Telco Customer Churn Classification

A Training-Only Workflow from Data Audit to Supervised Classification

Shakaar Latief

June 1, 2026

Abstract

This document reports a supervised classification project for the Telco Customer Churn dataset. The workflow audits the raw file, validates the target definition, handles deterministic data-quality issues, constructs a held-out train-test split, performs training-set exploratory analysis, defines a statistical evaluation methodology, establishes simple baselines, and then develops multiple model families. A central principle is that all modelling decisions are made using the training data and cross-validation only. The held-out test set remains reserved for final evaluation after model family, preprocessing, hyperparameters, threshold rule, and any calibration decisions are fixed. Section-level cross-validated results are therefore interpreted as development-stage estimates rather than final performance claims.

Contents

1 Raw data audit and deterministic correction

1.1 Purpose of the raw audit

This section documents the first workflow stage of the Telco Customer Churn classification project. The purpose is to inspect the raw dataset before constructing the modelling table and before creating the held-out train-test split.

The raw audit is deliberately limited. It does not estimate predictive patterns, compare feature distributions by churn status, create churn-rate plots, select features, tune preprocessing, or fit models. Instead, it answers the following questions:

- Does the raw file load correctly?
- What are the raw columns and data types?
- Are there duplicate rows?
- Are there standard missing values?
- Are there disguised missing values, such as blank strings?
- Does the target column exist and contain valid labels?
- Is there an identifier column that should be excluded from modelling?
- Are there deterministic raw-data representation problems that must be corrected before splitting?

This separation is important for leakage control. The full raw file may be inspected for schema and data-quality problems, because a valid modelling dataset cannot be created without those checks. However, feature-target exploratory analysis and any data-dependent modelling choices are postponed until after the train-test split and will use the training set only.

1.2 Raw dataset overview

The raw dataset contains 7043 observations and 21 columns. No exact duplicate rows were found. Pandas did not detect any standard missing values.

Table 1: Raw dataset overview

Item	Value
Number of rows	7043
Number of columns	21
Duplicate rows	0
Total standard missing values	0
Columns with standard missing values	0

The absence of standard missing values is not enough to conclude that no information is missing. String columns may contain blank strings or placeholder values that are not detected by ordinary missing-value checks.

1.3 Missing-data decision framework

Before choosing a cleaning strategy, it is useful to distinguish two different missing-data problems:

1. Missing feature values: at least one input variable X is missing.
2. Missing target labels: the output variable Y is missing.

These two cases should not be handled in the same way. For missing feature values, the central production question is whether missing inputs may occur when the model is used in the future. If missing feature values may occur in production, the final model pipeline must be able to handle them. In that setting, missing values should remain present in validation and test data so that evaluation simulates production. Any imputation rule or preprocessing rule must be fitted on the training data only and then applied unchanged to validation, test, and production data.

If production data is expected to be clean but the training data contains missing feature values, the goal is different: the experiment should simulate clean production. In that case, possible training-data strategies include removing incomplete training rows, imputing training values, removing a feature, or using model-based imputation. The choice should be made using training and validation logic, not the held-out test set.

For missing target labels, the rules are different. Missing labels in the training set may require training only on labelled observations, label imputation, or semi-supervised learning. Missing labels in the test set should not simply be imputed and treated as known. If test labels are missing, performance should be reported with uncertainty, for example through best-case and worst-case bounds.

In this dataset, the raw audit found no missing target labels and no standard missing feature values. It did find a disguised missing or representation issue in `TotalCharges`, which is discussed below.

1.4 Disguised missing values

The string-column audit checks for blank strings after trimming whitespace. This is necessary because blank strings such as `""` or `" "` are not automatically treated as missing values by pandas.

Table 2: Blank-string audit for selected string-like columns

Column	Blank string count	Blank string percentage	Unique values
TotalCharges	11	0.16	6531
customerID	0	0.00	7043
PaymentMethod	0	0.00	4
MultipleLines	0	0.00	3
InternetService	0	0.00	3
OnlineSecurity	0	0.00	3
OnlineBackup	0	0.00	3

Only `TotalCharges` contains blank strings. This is important because `TotalCharges` is conceptually numeric, but the blank strings cause the raw column to be read as string-like.

1.5 Target-label validation

The target variable is `Churn`. The observed target labels are `No` and `Yes`. There are no missing target labels and no unexpected target labels.

Table 3: Target distribution in the raw dataset

Target level	Count	Percentage
No	5174	73.46
Yes	1869	26.54

The positive class is defined as `Churn = Yes`. The target distribution is moderately imbalanced, with approximately 26.54% churn. This check justifies using a stratified train-test split in the next workflow stage. It is not used for model selection.

Table 4: Target-label validity check

Item	Value
Expected negative label present	True
Expected positive label present	True
Missing target values	0
Unexpected target labels	None

1.6 Identifier check

The column `customerID` has 7043 unique values for 7043 rows. It is therefore a unique identifier candidate. The identifier is useful for traceability, but it should not be used as a modelling feature because it does not describe a generalizable customer characteristic.

Table 5: Identifier check

Column	Unique values	Number of rows	Unique value ratio
customerID	7043	7043	1.00
TotalCharges	6531	7043	0.93
MonthlyCharges	1585	7043	0.23
tenure	73	7043	0.01
PaymentMethod	4	7043	0.00

1.7 Investigation of `TotalCharges`

The blank-string audit showed 11 blank strings in `TotalCharges`. These rows were checked against `tenure`. The result is exact: the 11 blank `TotalCharges` rows are exactly the 11 rows where `tenure = 0`.

Table 6: Relationship between blank `TotalCharges` and zero tenure

Condition	Count
Blank `TotalCharges`	11
`tenure == 0`	11
Blank `TotalCharges` and `tenure == 0`	11
Blank `TotalCharges` and `tenure != 0`	0
Non-blank `TotalCharges` and `tenure == 0`	0

Converting `TotalCharges` to numeric creates exactly 11 missing values, which are the same 11 blank-string records.

Table 7: `TotalCharges` numeric conversion check

Item	Count
Standard missing before conversion	0
Missing after numeric conversion	11
New missing created by numeric conversion	11
Blank string count	11

The affected rows are all zero-tenure customers. Most of these records have a two-year contract and none of them churned in the raw data. The churn outcome is not used to justify the correction. The correction is justified by the meaning of `TotalCharges`: a zero-tenure customer has not accumulated charges yet.

1.8 Deterministic interim correction

The following correction is applied to an interim copy of the raw dataset:

1. Convert `TotalCharges` from string-like values to numeric values.
2. For rows where `tenure = 0` and `TotalCharges` is blank after conversion, set `TotalCharges = 0.0`.

This is not mean imputation, median imputation, model-based imputation, or a target-informed correction. It is a deterministic representation correction based on the definition of accumulated charges. The original raw dataframe is kept unchanged, and the corrected copy is saved as the interim dataset for the next workflow stage.

After the correction, `TotalCharges` is numeric, contains no missing values, and ranges from 0.0 to 8684.8.

Table 8: Post-correction check

Item	Value
Remaining total missing values	0
Remaining missing `TotalCharges`	0
`TotalCharges` dtype	float64
Minimum `TotalCharges`	0.00
Maximum `TotalCharges`	8684.80

1.9 Basic numeric domain-validity checks

Unusual values are not automatically errors. A high but possible value may be a natural observation that belongs to the production distribution. However, values that violate basic domain constraints are different: they may indicate coding errors, measurement errors, or corrupted records.

The three numeric variables in this dataset should be non-negative:

- `tenure` is the number of months a customer has stayed with the company;
- `MonthlyCharges` is the customer's monthly charge amount;
- `TotalCharges` is the accumulated charge amount.

A value of zero can be valid. For example, zero tenure occurs for newly registered customers, and the corresponding accumulated total charge can be zero. Therefore, this audit checks only for negative values. This is not statistical outlier detection. Statistical outlier analysis is postponed until training-set exploratory analysis.

Table 9: Numeric domain-validity check after interim correction

Feature	Minimum	Maximum	Negative count	Negative percentage
tenure	0.00	72.00	0	0.00
MonthlyCharges	18.25	118.75	0	0.00
TotalCharges	0.00	8684.80	0	0.00

No negative values were found in the numeric variables. Therefore, no additional corrupted numeric values are corrected in this raw-audit stage.

1.10 Output of this workflow stage

The raw audit produces an interim dataset with 7043 rows, 21 columns, and no missing values. The raw target column `Churn` remains unchanged. The next workflow stage will create the binary target `Churn_binary`, exclude `customerID` from the modelling feature set, and create the held-out stratified train-test split.

Table 10: Interim dataset save check

Item	Value
Number of rows	7043
Number of columns	21
Total missing values	0
Interim file	`data/interim/telco_churn_interim.csv`

2 Clean modelling dataset and train-test split

2.1 Purpose of this stage

The raw-data audit produced an interim dataset in which the representation problem in `TotalCharges` was corrected. The next step is to create the supervised modelling table and the held-out train-test split.

This stage is deliberately narrow. It creates the binary target, defines the feature set, validates semantic feature groups, and creates a stratified split. It does not perform exploratory feature-target analysis, one-hot encoding, scaling, feature engineering, resampling, feature selection, or model fitting. Those steps are postponed until after the split and should use the training set only.

2.2 Interim dataset

The interim dataset is loaded from:

`data/interim/telco_churn_interim.csv`.

It has the same raw columns as the original dataset, except that `TotalCharges` has been converted to a numeric representation and the zero-tenure blank values have been set to `0.0`. No remaining missing values are present at this stage.

Table 11: Interim dataset overview

Item	Value
Number of rows	7043
Number of columns	21
Duplicate rows	0
Total missing values	0
Columns with missing values	0

2.3 Binary target encoding

The supervised learning target is `Churn_binary`. The positive class is churn, because this is the event of practical interest.

$$\texttt{Churn\_binary} = \begin{cases} 1, & \text{if } \texttt{Churn = Yes}, \\ 0, & \text{if } \texttt{Churn = No}. \end{cases}$$

The encoding produces no missing values and preserves the two-class structure of the original target.

Table 12: Target encoding

Original target level	Encoded value
No	0
Yes	1

2.4 Modelling feature set

The modelling feature set excludes `customerID`, the original string target `Churn`, and the binary target `Churn_binary`. The identifier is excluded because it is row-specific and does not represent a generalizable customer characteristic. The original target is excluded because the model uses the encoded binary target.

The resulting modelling table contains 7043 observations, 19 feature columns, and one target column.

Table 13: Clean modelling table overview

Item	Value
Rows	7043
Feature columns	19
Target columns	1
Identifier included as feature	No
Original string target included as feature	No

2.5 Semantic feature groups

The 19 modelling features are divided into semantic groups. These groups describe the clean base dataset; they do not yet define the final preprocessing for every model. Model-specific preprocessing is handled later inside training-only pipelines.

Table 14: Feature groups for modelling

Feature group	Count	Features
Numeric features	3	`tenure`, `MonthlyCharges`, `TotalCharges`
Binary categorical features	6	`SeniorCitizen`, `gender`, `Partner`, `Dependents`, `PhoneService`, `PaperlessBilling`
Nominal categorical features	10	`MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`, `Contract`, `PaymentMethod`

The feature-group validation confirms that every modelling feature belongs to exactly one group: no feature is missing from the groups, no grouped feature lies outside the modelling feature set, and no feature is assigned more than once.

2.6 Final checks before splitting

The final modelling table contains no missing values. Exact duplicate rows in the full interim data are absent. After excluding the identifier, duplicate feature-target combinations may exist, but this is not automatically a data-quality issue because multiple customers can genuinely share the same observed characteristics and churn label.

Table 15: Final checks before splitting

Item	Value
Missing values in modelling table	0
Duplicate full interim rows	0
Duplicate feature-target rows after excluding identifier	22

2.7 Target distribution before splitting

The target distribution is moderately imbalanced. The positive class, churn, represents approximately 26.5% of observations. This motivates a stratified train-test split. This target-only summary is used for split design rather than feature selection or model interpretation. No feature-target relationships are inspected before the train-test split.

Table 16: Target distribution before splitting

`Churn_binary`	Count	Percentage
0	5174	73.46
1	1869	26.54

2.8 Train-test split strategy

The clean modelling table is split into a training set and a held-out test set. The test set is reserved for final evaluation and should not be used for feature-target EDA, preprocessing design, model selection, hyperparameter tuning, or threshold tuning.

The split uses:

$$\texttt{test\_size} = 0.20, \qquad \texttt{random\_state} = 42.$$

The split is stratified by `Churn_binary`, so the train and test sets preserve the original churn proportion.

Table 17: Train-test split overview

Split	Rows	Percentage
Train	5634	79.99
Test	1409	20.01

2.9 Target distribution by split

The stratified split preserves the target distribution closely.

Table 18: Target distribution by split

Split	Churn_binary	Count	Percentage
Train	0	4139	73.46
Train	1	1495	26.54
Test	0	1035	73.46
Test	1	374	26.54

2.10 Saved outputs

The training and test datasets are saved locally as:

`data/processed/train.csv`, `data/processed/test.csv`.

These files form the boundary for the rest of the project. The next workflow stage should load only `train.csv` for exploratory analysis and modelling decisions. The held-out test file should remain unused until final model evaluation.

3 Exploratory data analysis

3.1 Purpose of this stage

The raw-data audit and splitting stages created a corrected interim dataset, a clean modelling table, and a held-out train-test split. This section performs exploratory data analysis using only the training set.

This restriction is methodological. Exploratory findings can influence later modelling choices, including feature engineering, transformations, rule-based baselines, model selection, outlier treatment, and interpretation. Therefore, feature distributions and feature-target relationships are not inspected on the held-out test set.

The goals of this stage are to understand the training data, identify modelling-relevant patterns, and produce report-ready figures and tables while preserving the test set for final evaluation. Large figures are split when needed for readability; a figure is kept in the main text when it is directly used for interpretation, while purely supplementary diagnostics can be moved to an appendix in later stages.

3.2 Training-set overview

The dataset used in this section contains 5634 observations. It has 19 feature columns and the binary target column `Churn_binary`. No missing values remain after the raw audit and deterministic `TotalCharges` correction.

Table 19: Training-set overview

Item	Value
Training rows	5634
Training columns	20
Missing values	0
Columns with missing values	0
Duplicate feature-target rows	15

The duplicate feature-target rows are not treated as a data-quality issue. The unique customer identifier was intentionally removed before modelling, so different customers can legitimately share the same observed feature values and churn label.

3.3 Target distribution

The positive class is churn, encoded as `1`. The target distribution remains moderately imbalanced, with churn representing 26.54% of observations.

Table 20: Target distribution

`Churn_binary`	Count	Percentage
0	4139	73.46
1	1495	26.54

This imbalance is not extreme, but it is large enough that accuracy alone will be an incomplete evaluation metric. Later modelling sections should include metrics that distinguish false positives from false negatives, such as precision, recall, specificity, F1-score, ROC-AUC, PR-AUC, and threshold analysis.

3.4 Numeric feature summary

The dataset has three numeric features: `tenure`, `MonthlyCharges`, and `TotalCharges`. Numeric summaries are computed on this development split only.

Table 21: Numeric feature summary

Feature	Count	Mean	Std.	Min	25%	Median	75%	Max
`tenure`	5634	32.49	24.57	0.00	9.00	29.00	55.00	72.00
`MonthlyCharges`	5634	64.93	30.14	18.40	35.66	70.50	90.00	118.75
`TotalCharges`	5634	2299.33	2279.20	0.00	402.98	1394.93	3835.83	8684.80

The numeric variables have different scales. This matters for later model families such as k-nearest neighbours, support vector machines, logistic regression with regularization, and neural networks, where scaling is usually required. Scaling should be fitted inside training-only pipelines rather than on the full dataset.

3.5 Numeric feature distributions

Figure 1 shows histograms and boxplots for the three numeric features in this split.

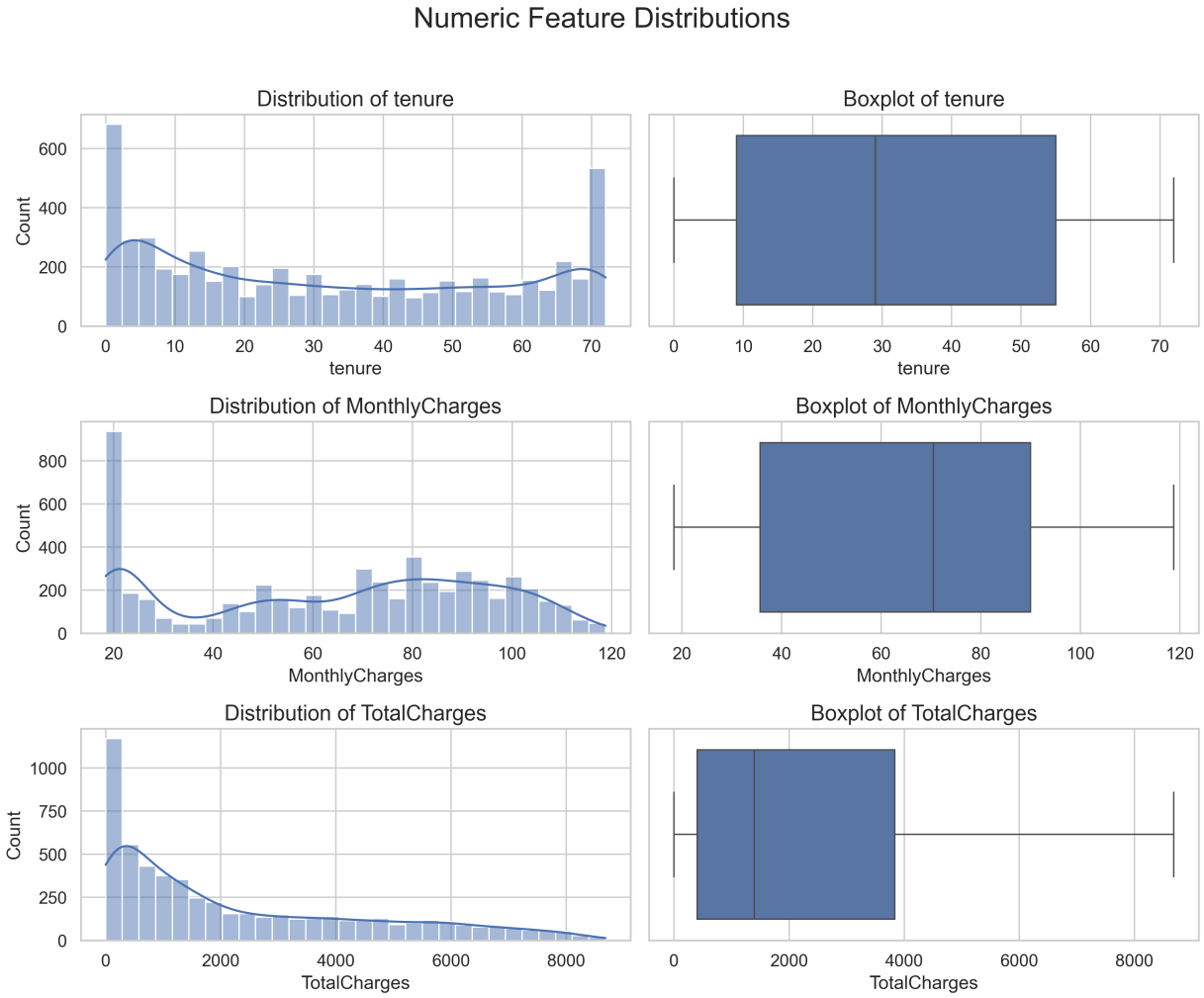

Figure 1: Numeric feature distributions and boxplots

The distribution of `tenure` has substantial mass near low values and also many observations near the upper end of the observed range. This indicates that the training set contains both newly joined customers and long-tenure customers.

The distribution of `MonthlyCharges` is not symmetric. There is a large group of customers with relatively low monthly charges and a broad group at higher monthly charge levels. The distribution of `TotalCharges` is strongly right-skewed. This is expected because total charges accumulate over time: many customers have low accumulated charges, while long-tenure or high-monthly-charge customers can have much larger accumulated totals.

These plots are descriptive. They do not justify automatic outlier removal. The raw audit already checked for impossible negative numeric values. High but plausible values should be treated as potential natural observations unless there is separate evidence that they are coding errors.

3.6 Numeric feature distributions by churn

Figure 2 compares the numeric feature distributions between churners and non-churners.

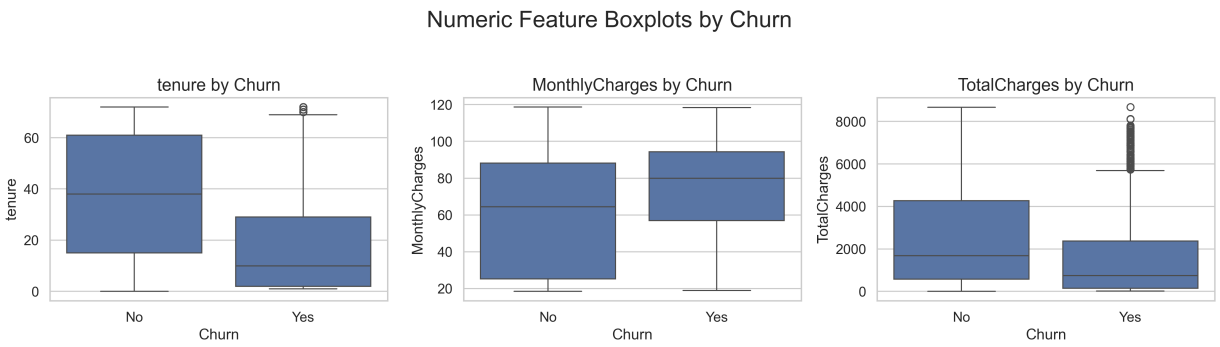

Figure 2: Numeric feature boxplots by churn

The class-conditioned boxplots show clearer feature-target structure than the marginal distributions. Customers who churned have substantially lower `tenure` than customers who did not churn. This is one of the strongest numeric patterns in the training data and suggests that newer customers are more likely to leave.

`MonthlyCharges` tends to be higher for churners. This indicates that customers with higher monthly bills may be at greater churn risk, at least as a marginal training-set association. `TotalCharges` tends to be lower for churners, but this should be interpreted carefully. Since `TotalCharges` accumulates over time, lower total charges among churners are consistent with churners having shorter tenure.

3.7 Numeric scatter matrix

Figure 3 shows a scatter matrix for the numeric features, coloured by churn status.

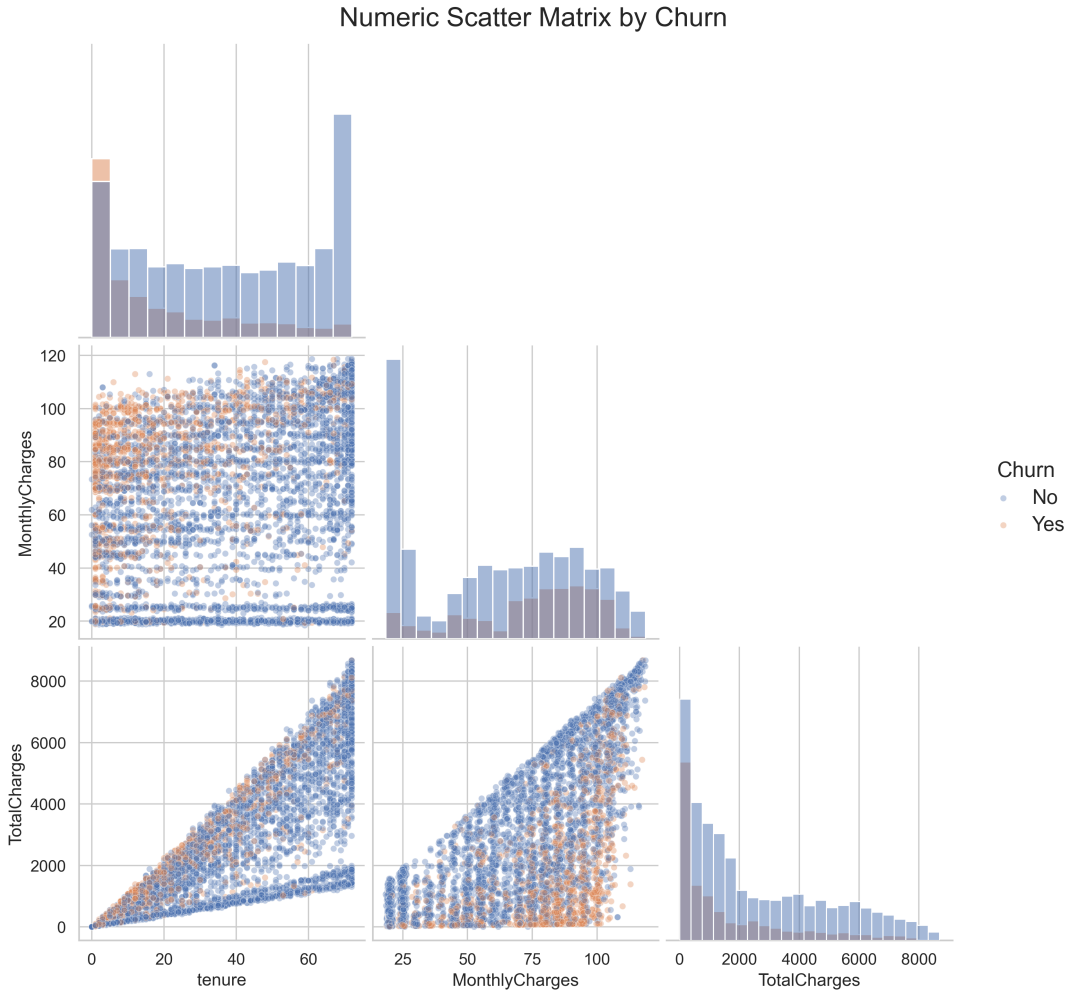

Figure 3: Numeric scatter matrix by churn

The scatter matrix shows visible churn-related patterns, but it does not show a clean visual separation between churners and non-churners using any single pair of numeric variables. Churners are more concentrated among customers with shorter tenure and are also relatively visible among customers with higher monthly charges. However, the two classes overlap substantially across the numeric feature space, suggesting that churn is not separable by a simple pairwise numeric boundary.

The clearest structural pattern is the relationship between `tenure` and `TotalCharges`. The triangular shape is expected because total charges accumulate over months and are also affected by monthly charges. For a fixed tenure, customers with higher monthly charges can accumulate higher total charges. This motivates using models that can combine numeric and categorical information, and possibly models that can learn interactions.

3.8 Numeric correlation matrix

Figure 4 shows the Pearson correlation matrix among numeric features and `Churn_binary` in this split.

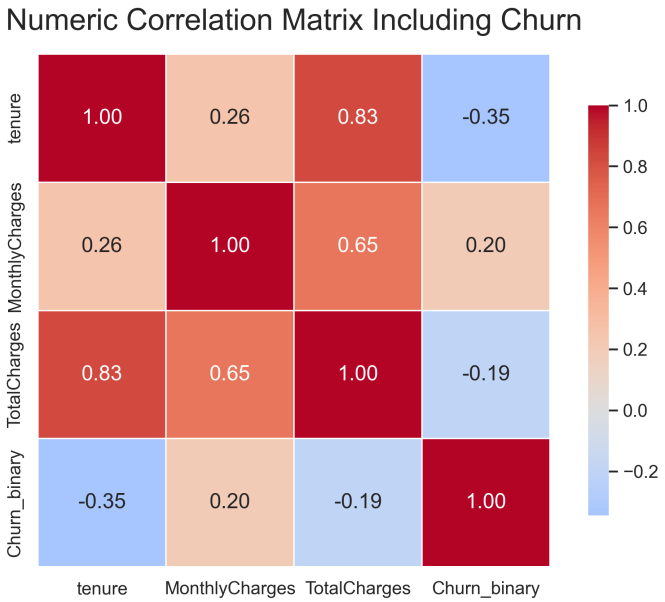

Figure 4: Numeric correlation matrix including `Churn_binary`

The strongest numeric correlation with churn is for `tenure`, with correlation approximately -0.35. Longer-tenure customers are less likely to churn in the training data. `MonthlyCharges` has a positive correlation with churn of approximately 0.20, while `TotalCharges` has a negative correlation with churn of approximately -0.19.

The correlation matrix also shows that `tenure` and `TotalCharges` are strongly correlated, approximately 0.83, and that `MonthlyCharges` and `TotalCharges` are also positively correlated, approximately 0.65. These are marginal linear associations only. They do not capture nonlinear relationships, interactions, or the effects of categorical variables.

3.9 Categorical feature frequencies

Figures 5 and 6 show the distribution of categorical feature levels. The categorical variables are split across two figures to keep axis labels and category names readable in the compiled report.

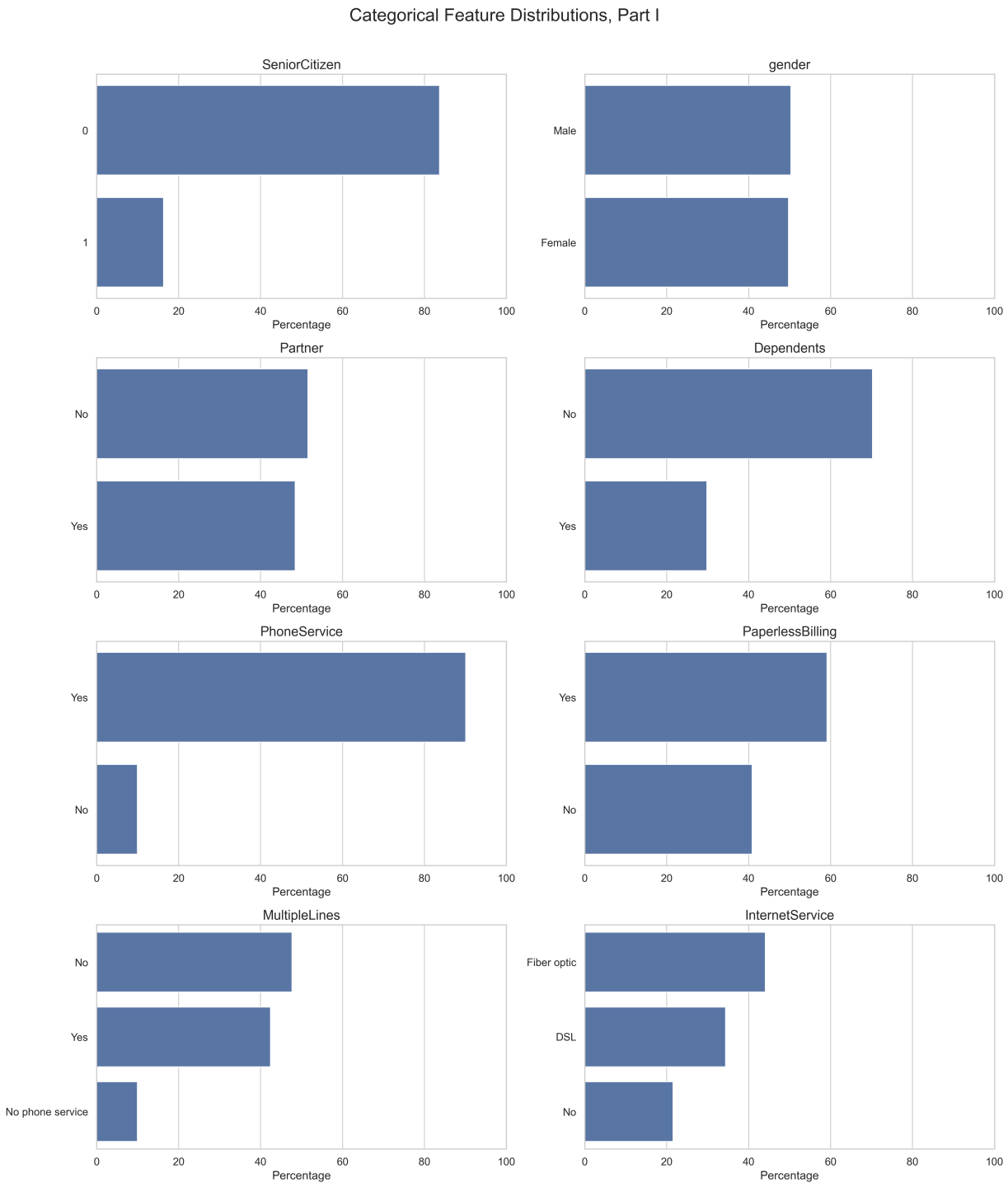

Figure 5: Categorical feature distributions, Part I

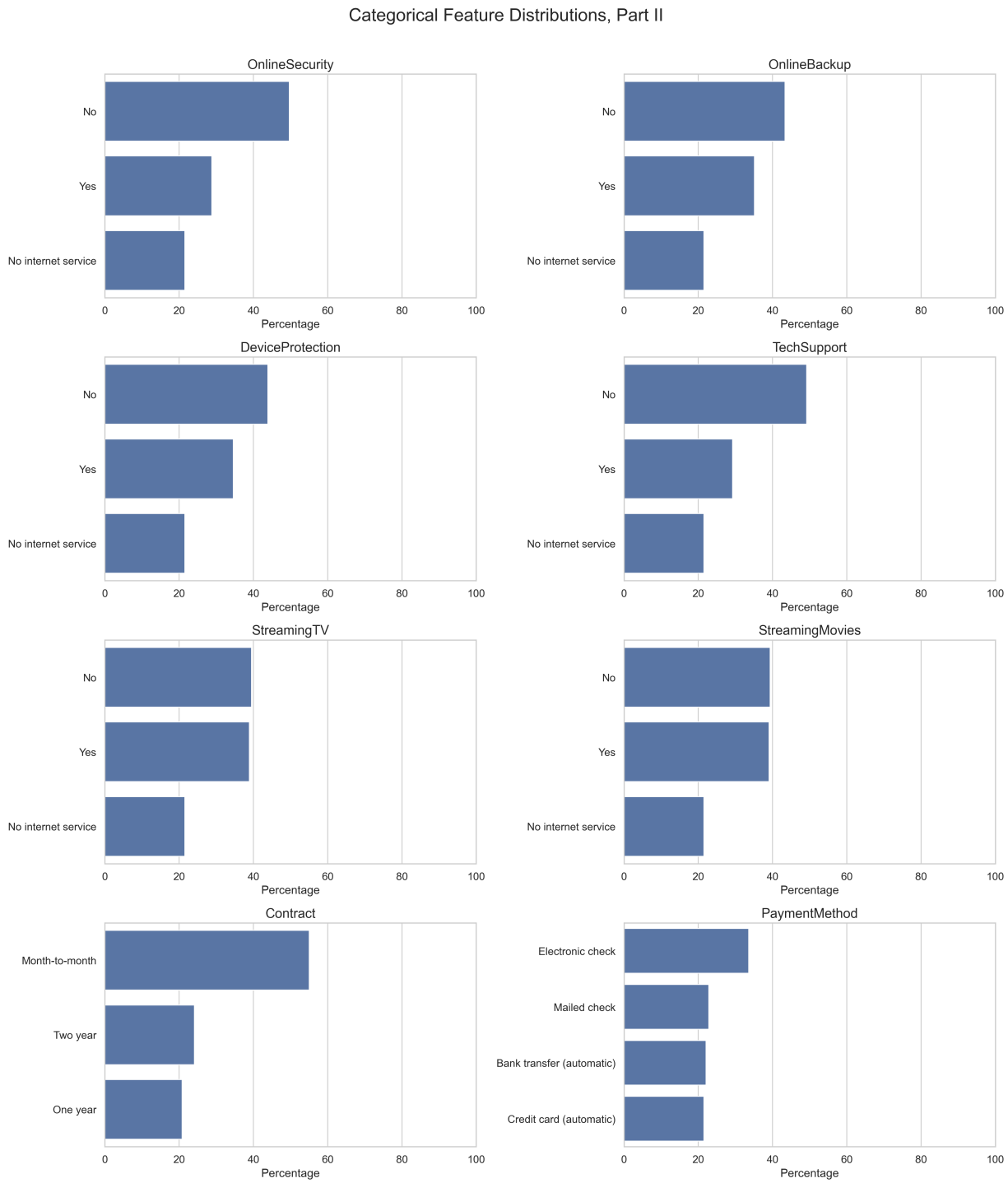

Figure 6: Categorical feature distributions, Part II

Several categorical predictors contain strongly imbalanced levels. For example, `PhoneService = Yes` represents 90.08% of the training set, `SeniorCitizen = 0` represents 83.67%, and `Contract = Month-to-month` represents 55.06%. These imbalances matter because rare levels can create unstable estimates in simple baselines and can affect how much information a model can learn from a category.

Several internet-service add-on variables contain the structural level `No internet service`. This is not missingness. It is an informative category indicating that the customer does not have internet service. Later preprocessing should preserve this category rather than treating it as a

missing value.

3.10 Categorical churn rates

Figures 7 and 8 show the churn rate within each categorical level. Both parts are included in the main section because they support the interpretation: Part I contains demographic, household, billing, phone-service, and internet-service variables, while Part II contains several service add-ons as well as contract type and payment method.

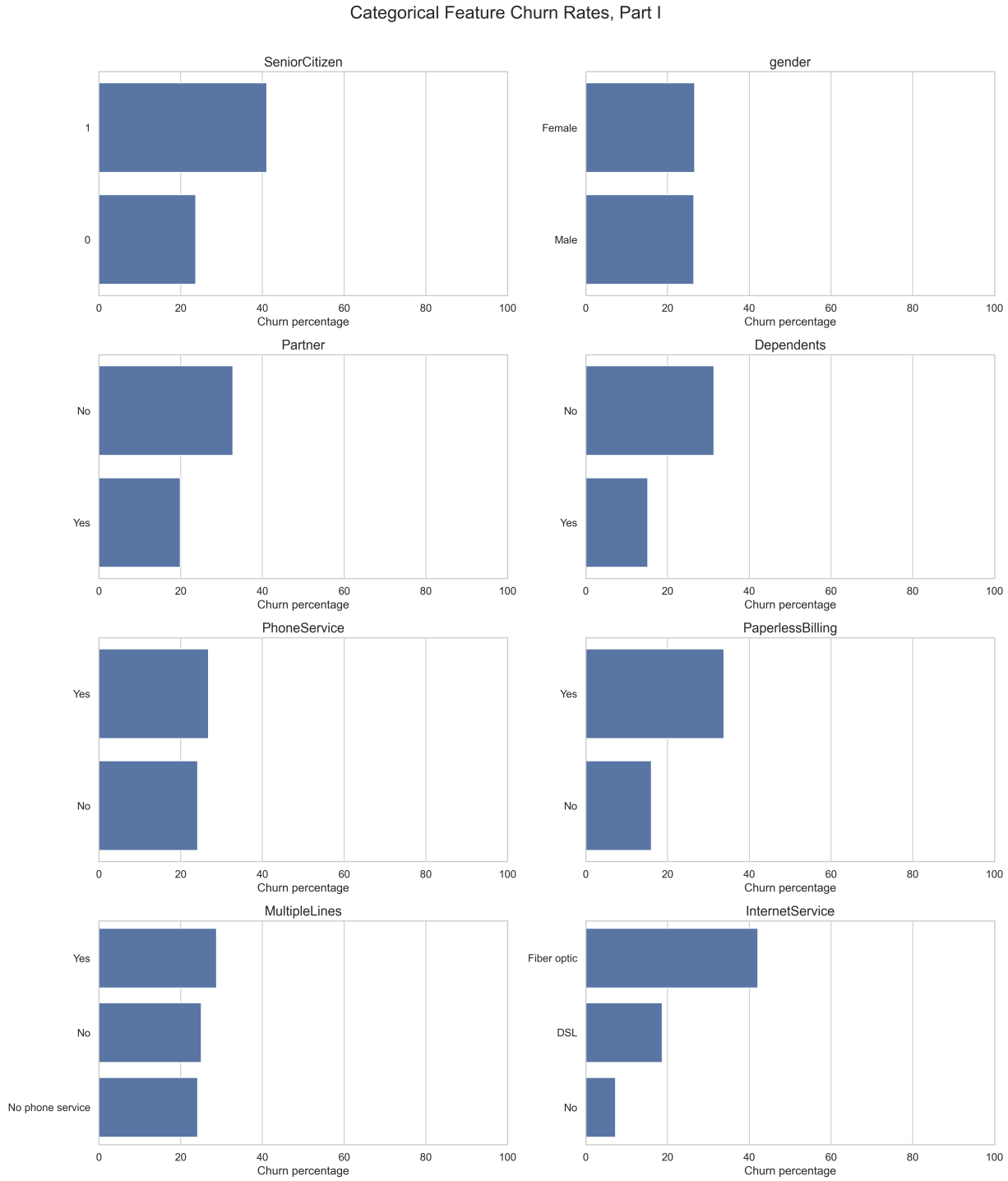

Figure 7: Categorical feature churn rates, Part I

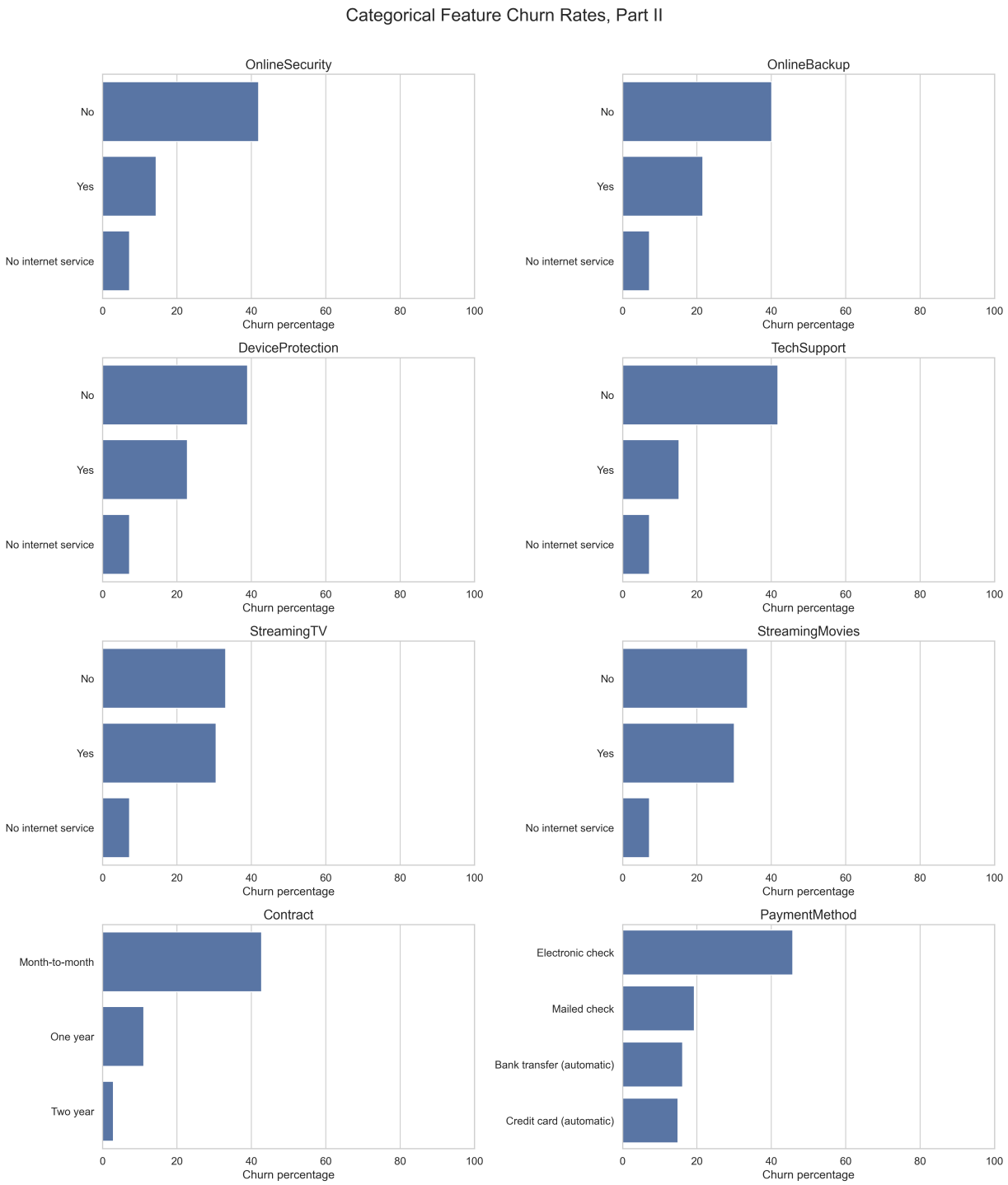

Figure 8: Categorical feature churn rates, Part II

The categorical churn-rate plots reveal several large training-set differences. Contract type is one of the strongest categorical patterns: month-to-month customers have a churn rate of 42.75%, compared with 11.08% for one-year contracts and 2.87% for two-year contracts. Internet service type also differs strongly: fiber optic customers have a churn rate of 42.09%, compared with 18.69% for DSL and 7.25% for customers with no internet service.

Payment method is also informative. Customers using electronic check have a churn rate of 45.74%, higher than mailed check at 19.28%, bank transfer at 16.16%, and credit card at 14.92%. Customers without online security or tech support also show much higher churn rates than

customers with those services.

Some variables show weak marginal separation. For example, churn rates for `gender` are almost identical, and `PhoneService` shows only a small difference. This does not mean these variables must be removed, because they may still contribute through interactions or in combination with other features. It does suggest that their standalone marginal relationship with churn is weak.

3.11 Selected categorical churn-rate table

Table 22 reports selected categorical levels that are especially relevant for interpretation and later baseline design.

Table 22: Selected categorical churn rates

Feature	Level	Count	Training %	Churn %
`Contract`	Month-to-month	3102	55.06	42.75
`Contract`	One year	1173	20.82	11.08
`Contract`	Two year	1359	24.12	2.87
`InternetService`	Fiber optic	2483	44.07	42.09
`InternetService`	DSL	1937	34.38	18.69
`InternetService`	No	1214	21.55	7.25
`PaymentMethod`	Electronic check	1891	33.56	45.74
`PaperlessBilling`	Yes	3331	59.12	33.80
`PaperlessBilling`	No	2303	40.88	16.02
`SeniorCitizen`	1	920	16.33	41.09
`SeniorCitizen`	0	4714	83.67	23.70

These are associations in the training data, not causal effects. They are useful for understanding the dataset, designing simple baselines, and interpreting later models.

3.12 Summary

The EDA suggests that the dataset contains meaningful predictive structure. Numerically, churn is most clearly associated with lower tenure and somewhat higher monthly charges. `TotalCharges` is lower among churners, but this is closely related to the shorter tenure of churned customers.

Categorically, the strongest visible churn-rate differences occur for contract type, internet service type, payment method, online security, tech support, paperless billing, and senior-citizen status. These patterns motivate the next stage: building preprocessing pipelines and baseline classifiers using only the training set.

4 Statistical evaluation methodology

This section defines the statistical interpretation of the model evaluations used throughout the project. It is placed before the learned-model sections because later tables compare multiple model families, hyperparameter settings, thresholds, and ranking metrics. Without a careful evaluation framework, it is easy to overinterpret small differences in cross-validated scores.

The main principle is that a reported metric is an estimate of an unobserved population quantity. A model does not have only a table of observed scores; it has an unknown true generalization performance under the data-generating process. The dataset provides finite-sample estimates of that performance. Therefore, model evaluation is a statistical estimation problem.

4.1 Data-generating distribution and true performance

Let (X, Y) denote a customer feature vector and the corresponding churn label. The supervised learning problem can be written as

$$(X, Y) \sim p(x, y),$$

where $p(x, y)$ is the unknown data-generating distribution. The observed dataset is a finite sample,

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}.$$

The goal is not to perform well only on the observed rows, but to perform well on future customers drawn from the same, or sufficiently similar, distribution.

For a fitted classifier h, the true accuracy is

$$A(h) = P(h(X) = Y) = \mathbb{E}_{(X,Y)\sim p}\left[\mathbb{1}\{h(X) = Y\}\right].$$

This quantity is not directly observable because $p(x, y)$ is unknown. On a finite evaluation sample, it is estimated by

$$\widehat{A}(h) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\{h(x_i) = y_i\}.$$

The same distinction applies to recall, precision, specificity, F_1, ROC-AUC, PR-AUC, calibration metrics, and expected cost. Each reported value is a finite-sample estimate of a population quantity.

This distinction is especially important when comparing models with close observed scores. A difference such as 0.658 versus 0.655 in PR-AUC is not automatically a meaningful population-level difference. It may reflect real model quality, sampling noise, fold-split randomness, hyperparameter-selection noise, or training randomness.

4.2 Accuracy as a statistical estimator

Accuracy has a simple statistical interpretation. For a fixed classifier h, define

$$Z_i = \mathbb{1}\{h(x_i) = y_i\}.$$

If the evaluation observations are independent draws from the population, then Z_i can be viewed as a Bernoulli variable with success probability equal to the true accuracy $A(h)$. The sample accuracy is the average of these Bernoulli variables,

$$\widehat{A}(h) = \frac{1}{n}\sum_{i=1}^{n} Z_i.$$

A rough standard error is

$$\widehat{\mathrm{SE}}(\widehat{A}) = \sqrt{\frac{\widehat{A}(1-\widehat{A})}{n}}.$$

A normal-approximation 95 percent confidence interval is

$$\widehat{A} \pm 1.96\sqrt{\frac{\widehat{A}(1-\widehat{A})}{n}}.$$

This formula is not the final uncertainty method used for all metrics in this project, but it gives the core intuition: evaluation-set size controls how precisely performance can be estimated.

For example, if an accuracy estimate is 0.80 on only 100 observations, the rough standard error is

$$\sqrt{\frac{0.8(0.2)}{100}} = 0.04,$$

so a rough 95 percent interval is approximately

$$0.80 \pm 1.96(0.04) = 0.80 \pm 0.078.$$

That interval is wide. The same point estimate on 10000 observations has standard error

$$\sqrt{\frac{0.8(0.2)}{10000}} = 0.004,$$

which gives a much narrower interval. Thus, evaluation data size can be as important as training data size when the aim is to make precise performance claims.

4.3 Why other classification metrics need care

Accuracy is a simple average of correct/incorrect indicators. Other metrics are more complicated. Recall and specificity are class-conditional proportions:

$$\text{Recall} = \frac{TP}{TP+FN}, \qquad \text{Specificity} = \frac{TN}{TN+FP}.$$

Precision is a proportion among predicted positives:

$$\text{Precision} = \frac{TP}{TP+FP},$$

where the denominator depends on the model's predictions. The F_1-score is a nonlinear combination of precision and recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

ROC-AUC and PR-AUC are ranking metrics computed from score distributions across thresholds. Their sampling uncertainty is not captured by the simple binomial formula for accuracy.

For this reason, candidate comparison should eventually use training-only uncertainty tools, such as paired bootstrap intervals on validation or cross-validation predictions. The final test evaluation should use bootstrap confidence intervals only for the single frozen final model. Section-level model comparisons before the final stage should therefore be interpreted as development evidence rather than final statistical proof.

4.4 Train, validation, and test roles

The project follows a strict separation between training, validation, and test roles:

- training data are used to estimate model parameters;
- validation data or cross-validation inside the training set are used to choose preprocessing, features, hyperparameters, thresholds, calibration methods, and model families;
- test data are reserved for final performance estimation after all choices are fixed.

The held-out test set is not used in the current model sections. This is intentional. If the test set is repeatedly checked during development, it becomes part of the model-selection process and can no longer provide an honest final estimate. Once test-set information has influenced modelling decisions, that mistake cannot be undone without obtaining a new independent test set.

In this project, all section-level model results are computed inside the training set using stratified cross-validation. They are development-stage estimates. Final performance will be evaluated later on the untouched held-out test set after the final model, threshold, and any calibration choices have been fixed.

4.5 Cross-validation as a development-stage estimator

In K-fold cross-validation, the training set is split into K folds. For fold k, the model is fitted on the other $K-1$ folds and evaluated on fold k. If $\widehat{M}^{(k)}$ is the metric on fold k, then the fold-mean cross-validation estimate is

$$\widehat{M}_{CV} = \frac{1}{K}\sum_{k=1}^{K}\widehat{M}^{(k)}.$$

For the churn task, stratified cross-validation is used so that each fold approximately preserves the churn rate. This matters because churn is the minority class.

Cross-validation is useful because it uses the training data efficiently and avoids relying on a single validation split. However, it is still part of model development. When the same cross-validation procedure is used to choose a model or hyperparameter setting, the selected cross-validated score may be mildly optimistic.

4.6 Fold-mean metrics and pooled out-of-fold metrics

Two related summaries are used in cross-validation:

- fold-mean metrics, where the metric is computed separately on each fold and then averaged;
- pooled out-of-fold metrics, where all out-of-fold predictions are collected and one metric is computed on the pooled prediction vector.

Pooled out-of-fold predictions are very useful for confusion matrices, threshold curves, ROC curves, precision-recall curves, and calibration plots. However, for nonlinear ranking metrics such as ROC-AUC and PR-AUC, the pooled out-of-fold AUC can differ from the mean of the fold-level AUC values because pooled ranking compares observations whose scores were produced by different fitted models.

The current model sections use pooled out-of-fold predictions for threshold and curve diagnostics. Later, when the project adds a dedicated model-comparison stage, it can store both fold-level and pooled out-of-fold summaries.

4.7 Hyperparameter tuning and selection optimism

Hyperparameter tuning changes the interpretation of cross-validation scores. For a fixed modelling setup, cross-validation estimates that setup's development-stage performance. But when many settings are tried and the best score is selected, the selected score is the maximum of several noisy estimates.

For example, suppose a kNN grid tries many values of k, two distance metrics, and two weighting schemes. Some settings may receive slightly lucky validation folds. The best observed configuration is likely to be both good and somewhat lucky. This is selection optimism.

Therefore, when a section says that a configuration is selected, the intended meaning is:

selected within the development grid according to the chosen metric.

It does not mean:

proven to be uniquely optimal in the population.

This distinction is especially important when neighbouring hyperparameter settings have very similar scores.

4.8 Repeated cross-validation

Repeated cross-validation repeats the fold construction several times with different random splits. For example, repeated 5-fold CV with 10 repeats produces 50 validation scores. This reduces dependence on one particular fold split and can make hyperparameter tuning more stable.

Repeated CV is useful for serious candidate models later in the project, especially if model rankings are close. However, repeated CV does not remove selection optimism completely. If the best setting is selected after many repeated-CV comparisons, the selected score is still the winner among several estimates.

Thus:

repeated CV improves stability,

whereas

nested CV evaluates a tuning procedure more honestly.

4.9 Nested cross-validation

Nested cross-validation separates tuning from evaluation. It has two loops:

- the inner loop chooses hyperparameters using only the outer-training data;
- the outer loop evaluates the resulting tuned procedure on held-out outer-validation folds.

For each outer fold:

1. split the development data into outer-training and outer-validation parts;

2. run inner cross-validation on the outer-training part to choose hyperparameters;
3. fit the selected configuration on the full outer-training part;
4. evaluate once on the outer-validation part.

Nested CV estimates the performance of a procedure, not one fixed hyperparameter setting. For example, a nested-CV estimate for kNN evaluates the procedure "choose k, distance, and weights by inner CV, then fit kNN with those selected settings." Different outer folds may choose different values of k. That is normal because the object being evaluated is the tuning procedure.

Nested CV is useful for comparing tuned model families, such as tuned logistic regression versus tuned kNN versus tuned random forest. The current project does not need nested CV inside every educational model section. Instead, nested CV can be considered later when the strongest model families are compared more formally.

4.10 Fair hyperparameter-search effort

Model comparisons can be unfair when one model receives much more tuning effort than another. For example, comparing default logistic regression with a heavily optimized boosting model would partly compare tuning effort rather than model-family quality.

Fair comparison principles include:

- use the same training and validation data;
- use the same primary selection metric;
- use comparable search effort where feasible;
- document the search spaces;
- avoid tuning the favourite model much more than the baselines;
- consider random search, Bayesian optimization, or Optuna-style search when search spaces are large.

This does not mean every model family requires identical compute. Some model families have more consequential hyperparameters than others. But the final comparison should be transparent about search effort.

4.11 Grid search, random search, and adaptive search

Grid search evaluates every combination from predefined hyperparameter values. It is simple and transparent, but inefficient when the hyperparameter space has many dimensions and only some parameters matter strongly.

Random search samples configurations from predefined distributions. It is often more efficient in high-dimensional spaces because it explores more distinct values of the important dimensions for the same number of trials.

Adaptive methods such as Bayesian optimization or Optuna-style search use earlier trials to decide which configurations to try next. They can be useful for expensive model families, but they are less transparent and add another layer of procedure choice. For this project, simple grids are

suitable for early educational sections. More advanced search may be introduced later for final candidate models.

4.12 Threshold selection as model selection

Many classifiers output scores or probabilities. A threshold turns a score into a hard prediction:

$$\hat{y}_\tau = \mathbb{1}\{s(x) \geq \tau\}.$$

Changing τ changes recall, precision, specificity, F_1, predicted positive rate, and expected cost. Therefore, threshold choice is a model-selection decision.

Threshold curves in the model sections are diagnostic. They show how a model behaves across operating points. The final threshold should be chosen later using training/validation information only, then frozen before the test set is evaluated.

4.13 Calibration as a separate evaluation dimension

Ranking performance and probability calibration are different. ROC-AUC and PR-AUC measure whether the model ranks churners above non-churners. Calibration asks whether predicted probabilities are numerically reliable. A model can rank customers well but still produce overconfident or underconfident probabilities.

Calibration matters if predicted probabilities are interpreted as churn risks or used in cost-based decision rules. If calibration is applied later, the calibrator must also be fitted without test leakage, using validation data or a cross-validation-based calibration procedure.

4.14 Statistical tests and uncertainty methods

The final project can use several uncertainty tools, each answering a different question:

- bootstrap confidence intervals estimate uncertainty around final test metrics;
- paired bootstrap intervals can compare metric differences between candidate models before final selection, using validation or cross-validation predictions;
- McNemar's test compares paired hard-classification errors;
- DeLong-style tests compare paired ROC-AUCs;
- permutation tests can assess whether model performance is stronger than expected under random label association;
- repeated CV and nested CV can describe robustness and procedure-level performance before final test evaluation.

Bootstrap is especially useful because it can be applied to many metrics, including PR-AUC, ROC-AUC, F_1, recall, precision, balanced accuracy, Brier score, and expected cost. In the final test stage, the project should report point estimates together with uncertainty intervals for the single frozen final model wherever feasible.

4.15 Implications for the current report

The model sections that follow should be read as model-development sections. They explain model families and use training-set cross-validation to compare configurations. Their selected models are representative strong candidates within the tried development grids. They are not final claims of statistical superiority.

The report therefore uses the following interpretation:

- large differences are treated as useful development evidence;
- small differences between neighbouring hyperparameters are interpreted cautiously;
- selected configurations are described as selected within a development grid;
- threshold results are diagnostic rather than final operating rules;
- final model-family comparison, statistical tests, threshold selection, and calibration decisions are deferred to later training-only stages. Test-set performance is deferred until exactly one final model has been selected and frozen.

This discipline lets the project remain both educational and statistically careful. The individual sections can explain each model in depth, while the final model comparison stage can later use repeated CV, nested CV, bootstrap confidence intervals, paired comparisons, and ablation studies to support stronger final conclusions.

4.16 Strict final test-set policy

The final test set is reserved for exactly one frozen final model. It should not be used to compare candidate models, additional candidate models, alternative thresholds, alternative calibration methods, or alternative preprocessing decisions. All model comparisons and statistical tests used for selection should be completed before the final test set is touched, using training-set validation, repeated cross-validation, nested cross-validation, or other training-only procedures.

After the final model, preprocessing, hyperparameters, threshold, and calibration decision are frozen, the model is fitted on the full training set and evaluated once on the untouched test set. Bootstrap confidence intervals may then be computed for the metrics of that single final model. They are used to quantify finite-test-sample uncertainty, not to choose among models.

5 Preprocessing, evaluation, and simple baselines

5.1 Purpose of this stage

This section starts the modelling workflow after the raw-data audit, deterministic correction, train/test split, and training-set exploratory analysis. The purpose is not yet to fit flexible learned models. Instead, this stage establishes the evaluation language and simple reference baselines that all later models must improve upon.

The held-out test set remains unused. All results in this section are computed only from the training set using stratified cross-validation. This preserves the final test set for the final evaluation stage.

The section has three goals. First, it defines the binary classification evaluation framework used throughout the rest of the project. Second, it introduces reusable preprocessing infrastructure for later models. Third, it evaluates simple baselines that do not learn a flexible relationship from the full feature matrix.

The simple baselines considered here are:

1. a majority-class baseline;
2. a prior-probability baseline;
3. a stratified random baseline;
4. a uniform random baseline;
5. an exploratory-analysis-inspired rule baseline.

Learned models such as least-squares classification, logistic regression, k-nearest neighbours, Naive Bayes, decision trees, support vector machines, and neural networks are introduced in later sections.

5.2 Training-only evaluation discipline

Let the training set be

$$\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^{n},$$

where x_i denotes the feature vector for customer i, and $y_i \in \{0, 1\}$ denotes the binary churn label. In this project, the positive class is churn:

$$y_i = 1 \quad \Longleftrightarrow \quad \text{customer } i \text{ churned.}$$

All model-development decisions must be made without using the held-out test set. This includes preprocessing design, feature engineering, feature selection, model-family choice, hyperparameter tuning, threshold tuning, calibration decisions, and model comparison.

For this stage, performance is estimated using stratified 5-fold cross-validation inside the training set. Stratification preserves approximately the same churn rate in each fold. This is important because churn is the minority class. In the training set, the class distribution is shown in Table 23.

Table 23: Training-set target distribution.

Class	Count	Percentage
No churn ($Y = 0$)	4139	73.46%
Churn ($Y = 1$)	1495	26.54%

Cross-validation produces out-of-fold predictions. This means that the prediction for each observation is generated by a model that was not fitted on that same observation. This gives a more honest development-stage estimate than evaluating a model on the same data used to fit it.

5.3 Positive and negative classes

Many binary classification metrics are defined relative to the positive class. The positive class is not necessarily the most common class. It is the event of interest.

A medical screening example makes the terminology intuitive. Suppose a test predicts whether a person has cancer. Then:

- positive class: the person has cancer;
- negative class: the person does not have cancer;
- predicted positive: the test says cancer;
- predicted negative: the test says no cancer.

For the churn problem, the same logic becomes:

- positive class: the customer churns;
- negative class: the customer does not churn;
- predicted positive: the model flags the customer as likely to churn;
- predicted negative: the model does not flag the customer as likely to churn.

This convention matters because recall, precision, F_1, ROC-AUC, and PR-AUC all depend on which class is treated as positive.

5.4 Confusion matrix

The confusion matrix counts the four possible combinations of actual class and predicted class. The positive-first layout used here is shown in Table 24. Rows represent the actual class and columns represent the predicted class.

Table 24: Positive-first binary confusion matrix layout.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
Actual	Negative	FP	TN

The entries are:

$$TP = \#\{\hat{y} = 1, y = 1\}, \qquad FN = \#\{\hat{y} = 0, y = 1\},$$
$$FP = \#\{\hat{y} = 1, y = 0\}, \qquad TN = \#\{\hat{y} = 0, y = 0\}.$$

This order is chosen to match the lecture-style visual explanation: actual positives are shown first, and within that row the model can either correctly predict positive or incorrectly predict negative. Some software libraries store confusion matrices in a negative-first numeric order. In the code, the counts are computed with explicit labels and then renamed so that the report can present the matrix in the positive-first order.

In the medical screening example:

- true positive: the test says cancer, and the person really has cancer;
- false negative: the test says no cancer, but the person really has cancer;
- false positive: the test says cancer, but the person does not have cancer;
- true negative: the test says no cancer, and the person does not have cancer.

In the churn problem:

- true positive: the model predicts churn, and the customer churns;
- false negative: the model predicts no churn, but the customer churns;
- false positive: the model predicts churn, but the customer does not churn;
- true negative: the model predicts no churn, and the customer does not churn.

False positives and false negatives have different practical meanings. A false positive may lead to unnecessary retention effort for a customer who would have stayed anyway. A false negative may mean missing a customer who actually leaves.

5.5 Accuracy and the class-imbalance problem

Accuracy is the fraction of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}.$$

Accuracy is intuitive, but it can be misleading when one class is much more common than the other. In this dataset, about 73.46% of training customers do not churn. A classifier that always predicts no churn can therefore achieve 73.46% accuracy while detecting zero churners.

This is the central reason why the project does not evaluate churn models using accuracy alone. The evaluation must also measure how well the model detects the positive class and how many false churn alerts it creates.

5.6 Recall, specificity, precision, and F_1

Recall, also called true positive rate, measures the fraction of actual positives that are detected:

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN}.$$

In this project, recall answers: among all customers who churned, how many did the model flag?

Specificity, also called true negative rate, measures the fraction of actual negatives correctly rejected:

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}.$$

In this project, specificity answers: among all customers who did not churn, how many did the model correctly leave unflagged?

Precision measures the reliability of positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

In this project, precision answers: among all customers flagged as churners, how many actually churned?

The F_1-score combines precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

It is useful as a compact summary when both precision and recall matter. However, it hides the separate business meanings of false positives and false negatives. For this reason, F_1 is reported together with the underlying precision and recall values.

5.7 Balanced accuracy

Balanced accuracy averages recall and specificity:

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\text{Recall} + \text{Specificity}\right).$$

Balanced accuracy is useful under class imbalance because it gives equal importance to the positive and negative classes. A classifier that always predicts one class receives balanced accuracy equal to 0.5 in a binary problem, because it perfectly classifies one class and completely fails on the other.

This makes balanced accuracy useful for comparing simple baselines. A majority-class classifier may have high ordinary accuracy, but its balanced accuracy reveals whether it is actually useful for both classes.

5.8 ROC-AUC and PR-AUC

Many classifiers produce a score or probability rather than only a hard class label. A threshold turns this score into a class prediction:

$$\hat{y}_\tau = \mathbb{1}\{s(x) \geq \tau\},$$

where $s(x)$ is a model score or predicted probability and τ is the decision threshold.

Changing the threshold changes the tradeoff between false positives and false negatives. ROC and precision-recall curves summarize this threshold-dependent behaviour.

The ROC curve plots:

$$\text{FPR} = \frac{FP}{FP + TN} \quad \text{against} \quad \text{TPR} = \frac{TP}{TP + FN}.$$

ROC-AUC summarizes how well the model ranks positive cases above negative cases across thresholds.

The precision-recall curve plots:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{against} \quad \text{Recall} = \frac{TP}{TP + FN}.$$

PR-AUC summarizes how precise the positive predictions remain as the model attempts to recover more positives.

ROC-AUC and PR-AUC often move in the same direction when one model is clearly better at ranking positive cases. They can differ under class imbalance. A model can have a reasonable ROC-AUC while still having low precision if many of its predicted positives are false positives. Since churn is the minority class and the positive class is the main event of interest, PR-AUC is an important complement to ROC-AUC.

5.9 Thresholds and calibration

A probabilistic classifier estimates:

$$\hat{p}(Y = 1 \mid X = x).$$

The default hard classification rule is often:

$$\hat{y} = \mathbb{1}\{\hat{p}(Y = 1 \mid X = x) \geq 0.5\}.$$

The threshold 0.5 is not automatically optimal. In churn prediction, the preferred threshold depends on the relative cost of offering retention actions to customers who would not churn versus missing customers who will churn.

Calibration is a separate question. A model is calibrated if predicted probabilities behave like empirical probabilities:

$$P(Y = 1 \mid \hat{p} = q) \approx q.$$

For example, among customers assigned a churn probability near 0.30, about 30% should actually churn. Calibration will become more important when learned probability models are introduced.

5.10 Class imbalance strategies

The current section evaluates baselines under the natural training distribution. Resampling strategies such as random oversampling, random undersampling, and SMOTE are not applied yet because they are training interventions for learned models rather than baseline definitions.

The validation and test distributions should remain realistic. If resampling is used later, it must be applied only inside the training part of each cross-validation split. The validation fold must remain untouched. The incorrect workflow is:

$$\text{resample full training set} \quad \rightarrow \quad \text{cross-validate on resampled data.}$$

This can leak information or artificially improve validation performance because the validation fold no longer represents an untouched natural sample.

The correct workflow is:

$$\text{split fold} \quad \rightarrow \quad \text{fit preprocessing on training fold} \quad \rightarrow \quad \text{resample training fold only} \quad \rightarrow \quad \text{fit model} \quad \rightarrow \quad \text{e}$$

In Python, this usually requires an imbalanced-learn pipeline rather than an ordinary scikit-learn pipeline, because the sampler changes both X and y. Resampling, class weighting, and threshold tuning are considered later after learned models have been introduced.

5.11 Reusable preprocessing infrastructure

Most simple baselines in this section do not require feature preprocessing. Dummy classifiers ignore the feature matrix, and the rule-based classifier works directly with a small number of original categorical variables.

Nevertheless, reusable preprocessing objects are introduced here because later model sections depend on them. Two broad preprocessing variants are prepared:

- a scaled preprocessor for scale-sensitive models such as logistic regression, k-nearest neighbours, support vector machines, and neural networks;
- an unscaled preprocessor for tree-based models, where numeric scaling is usually unnecessary.

Categorical variables are one-hot encoded in standard modelling pipelines. Structural categories such as `No internet service` and `No phone service` are treated as valid categories, not as missing values.

5.12 Simple baseline definitions

The majority-class baseline always predicts the most frequent class observed in the training fold:

$$\hat{y} = \arg\max_{c \in \{0,1\}} \sum_{i=1}^{n} \mathbb{1}\{y_i = c\}.$$

In this dataset, this means always predicting no churn.

The prior-probability baseline uses the empirical class probabilities:

$$\hat{p}(Y = c) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}\{y_i = c\}.$$

Its hard class prediction is still the majority class, but its predicted probabilities correspond to the class prior.

The stratified random baseline randomly samples labels according to the training-fold class distribution:

$$\hat{Y} \sim \text{Categorical}\left(\hat{p}(Y = 0), \hat{p}(Y = 1)\right).$$

The uniform random baseline samples labels uniformly:

$$\hat{Y} \sim \text{Categorical}(0.5, 0.5).$$

The exploratory-analysis-inspired rule baseline assigns one risk point for each high-risk condition:

$$s(x) = \sum_{j=1}^{m} \mathbb{1}\{\text{condition } j \text{ is true}\}.$$

The predicted class is:

$$\hat{y} = \mathbb{1}\{s(x) \geq t\}.$$

The rule used here sets $t = 2$, with the following high-risk conditions:

- month-to-month contract;
- electronic check payment;
- fiber optic internet service;
- no online security;
- no tech support.

This rule is intentionally simple. It is not a final model. Its purpose is to test whether the strongest training-set EDA patterns already contain predictive signal.

5.13 Simple baseline results

Table 25 reports the cross-validated metrics for the simple baselines. Table 26 reports the corresponding confusion-matrix counts in the positive-first order TP, FN, FP, TN.

Table 25: Cross-validated simple-baseline metrics on the training set.

Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	F_1	ROC-AUC	PR-AUC	Pred. pos.	Obs. pos.
EDA-inspired rule	0.6076	0.7032	0.3956	0.9070	0.4994	0.5509	0.8109	0.5435	0.6084	0.2654
Dummy: stratified random	0.6140	0.5065	0.2748	0.2776	0.7354	0.2762	0.5065	0.2680	0.2680	0.2654
Dummy: uniform random	0.5051	0.5057	0.2698	0.5070	0.5045	0.3522	0.5000	0.2654	0.4986	0.2654
Dummy: majority class	0.7346	0.5000	0.0000	0.0000	1.0000	0.0000	0.5000	0.2654	0.0000	0.2654
Dummy: prior probability	0.7346	0.5000	0.0000	0.0000	1.0000	0.0000	0.4999	0.2653	0.0000	0.2654

Table 26: Cross-validated confusion-matrix counts for the simple baselines, shown in positive-first order.

Model	TP	FN	FP	TN
EDA-inspired rule	1356	139	2072	2067
Dummy: stratified random	415	1080	1095	3044
Dummy: uniform random	758	737	2051	2088
Dummy: majority class	0	1495	0	4139
Dummy: prior probability	0	1495	0	4139

5.14 Interpretation

The majority-class and prior-probability baselines have the highest ordinary accuracy among the simple baselines, 0.7346. However, this accuracy is misleading. Both baselines predict no churn for every customer. Therefore, they classify all 4139 non-churners correctly but miss all 1495 churners. Their recall is exactly 0.0000, and their balanced accuracy is only 0.5000.

The stratified random baseline predicts churn at approximately the observed training-set churn rate. Its predicted positive rate is 0.2680, close to the observed positive rate 0.2654. Its balanced accuracy is 0.5065, which is only slightly above random-level performance. This is expected because the predictions are random and not informed by the features.

The uniform random baseline predicts churn for about half of the customers. This gives recall 0.5070, higher than the majority-class baseline, but this is achieved by producing many false positives. The model incorrectly flags 2051 non-churners as churners. Its ROC-AUC is 0.5000, as expected for a random classifier.

The EDA-inspired rule is the only simple baseline that uses feature information. It detects 1356 of the 1495 churners, giving recall 0.9070. This confirms that the strongest EDA patterns contain substantial predictive signal. However, it also incorrectly flags 2072 non-churners, so its specificity is only 0.4994 and its precision is 0.3956.

This makes the EDA-inspired rule a high-recall, low-specificity baseline. It is useful because it shows that simple churn-risk conditions already recover most churners. It is not sufficient as a final model because it predicts churn for about 60.84% of customers, much higher than the observed churn rate of 26.54%. A practical learned model should aim to keep much of this recall while improving precision and specificity.

5.15 Implications for later models

The simple baselines establish the minimum standard for later learned models. A useful learned model should:

- clearly outperform dummy baselines;
- avoid relying only on ordinary accuracy;
- improve balanced accuracy relative to random baselines;
- detect churners with non-trivial recall;
- improve precision and specificity relative to the broad EDA rule;
- provide useful ranking performance through ROC-AUC and PR-AUC;
- eventually support threshold tuning and calibration analysis.

The next modelling stage begins the learned-model sequence with linear classification, least-squares classification, logistic regression, and regularized logistic regression.

6 Linear classification and logistic regression

6.1 Purpose of this stage

This stage introduces the first learned classification models in the project. The previous stages established the statistical evaluation methodology, the binary classification evaluation framework, simple dummy baselines, and an EDA-inspired rule. The purpose here is to move from manually specified or non-informative baselines to learned linear classifiers that estimate decision rules from the training data.

The held-out test set is not used in this section. All performance estimates are based on stratified cross-validation inside the training set. Preprocessing is included inside each modelling pipeline, so scaling and one-hot encoding are fitted only on the training portion of each cross-validation fold.

The models considered in this section are regularized least-squares classification, logistic regression with L2 regularization, logistic regression with L1 regularization, and a class-weighted logistic regression variant. These models are useful first learned models because they are mathematically transparent, computationally efficient, and interpretable through their fitted coefficients.

The results in this section should be read as development-stage cross-validated estimates. They are used to understand the behaviour of linear classifiers and to choose representative linear candidates for later comparison. They are not final test-set estimates, and small differences between neighbouring regularization settings should not be interpreted as proof of unique population-level optimality.

6.2 Linear score functions and decision boundaries

A linear classifier begins with a real-valued score

$$f(x) = w^\top x + b,$$

where $x \in \mathbb{R}^p$ is a feature vector, $w \in \mathbb{R}^p$ is a vector of weights, and $b \in \mathbb{R}$ is an intercept. The score is positive on one side of the decision boundary and negative on the other side.

The decision boundary is the set of feature values for which the score equals zero:

$$w^\top x + b = 0.$$

In two dimensions this boundary is a line, in three dimensions it is a plane, and in higher dimensions it is a hyperplane. A simple binary prediction rule is

$$\hat{y} = \begin{cases} 1, & f(x) \geq 0, \\ 0, & f(x) < 0. \end{cases}$$

In this project, 1 denotes churn and 0 denotes no churn. Therefore, a positive score assigns a customer to the churn side of the fitted linear boundary.

Linear classifiers are limited because they are additive in the transformed feature representation. After one-hot encoding, they can learn separate contributions from contract type, internet service, billing, and numeric variables, but they do not automatically learn complex interactions unless those interactions are explicitly added as features. This limitation is useful to remember when later comparing linear models with trees, ensembles, and neural networks.

6.3 Least-squares classification

Least-squares classification treats classification as a regression-like problem. If labels are encoded as $y_i^\star \in \{-1, +1\}$, the fitted score is obtained by minimizing

$$\min_{w,b} \sum_{i=1}^{n} \left(w^\top x_i + b - y_i^\star \right)^2 .$$

In matrix form, with the intercept included in the design matrix, this is

$$\min_{\beta} \|X\beta - y^\star\|_2^2.$$

A regularized version adds an L2 penalty:

$$\min_{\beta} \|X\beta - y^\star\|_2^2 + \alpha\|\beta\|_2^2.$$

This is the idea behind the `RidgeClassifier` used in this section. It provides a linear classification baseline connected to least-squares learning, while the ridge penalty stabilizes fitting after one-hot encoding.

Least-squares classification is not a probability model. It can produce a useful linear score, but it does not directly model $P(Y = 1 \mid X = x)$. Logistic regression is more appropriate when probability estimates and threshold analysis are needed.

6.4 Logistic regression

Logistic regression uses the same linear score,

$$z_i = w^\top x_i + b,$$

but maps it to a probability through the sigmoid function:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

The fitted churn probability is

$$\hat{p}_i = P(Y_i = 1 \mid X_i = x_i) = \sigma(w^\top x_i + b).$$

The default class prediction uses threshold 0.5:

$$\hat{y}_i = \mathbb{1}\{\hat{p}_i \geq 0.5\}.$$

Because the sigmoid is monotonic and $\sigma(0) = 0.5$, this default threshold corresponds to the linear boundary $w^\top x + b = 0$.

Logistic regression can also be written as a log-odds model:

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = w^\top x + b.$$

A coefficient w_j is therefore a change in log-odds associated with a one-unit increase in the transformed feature x_j, holding other transformed features fixed. The odds ratio is

$$\exp(w_j).$$

For standardized numeric variables, a one-unit change corresponds to a one-standard-deviation increase in the original numeric feature. For one-hot encoded variables, coefficients describe the contribution of the corresponding category indicator within the fitted linear model. These are fitted associations, not causal effects.

6.5 Likelihood and log loss

For binary outcomes, logistic regression assumes

$$Y_i \mid X_i = x_i \sim \text{Bernoulli}(\hat{p}_i).$$

The probability of observing y_i is

$$P(Y_i = y_i \mid x_i) = \hat{p}_i^{y_i}(1 - \hat{p}_i)^{1-y_i}.$$

The log-likelihood is

$$\ell(w, b) = \sum_{i=1}^{n} \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right].$$

Equivalently, logistic regression minimizes the negative log-likelihood,

$$\mathcal{L}(w, b) = -\sum_{i=1}^{n} \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right],$$

also called binary cross-entropy or log loss. This loss strongly penalizes confident wrong predictions. For example, assigning a very low probability to a true churner gives a large loss contribution.

6.6 Regularization

Regularization controls model complexity by penalizing large coefficients. L2-regularized logistic regression minimizes

$$\mathcal{L}_{L2}(w, b) = \mathcal{L}(w, b) + \lambda \|w\|_2^2,$$

where

$$\|w\|_2^2 = \sum_{j=1}^{p} w_j^2.$$

L2 regularization shrinks coefficients toward zero but usually does not set them exactly to zero.

L1-regularized logistic regression minimizes

$$\mathcal{L}_{L1}(w, b) = \mathcal{L}(w, b) + \lambda \|w\|_1,$$

where

$$\|w\|_1 = \sum_{j=1}^{p} |w_j|.$$

L1 regularization can set coefficients exactly to zero, making it useful for sparse linear models and feature selection.

In scikit-learn, logistic regression uses C as inverse regularization strength. Smaller C means stronger regularization, while larger C means weaker regularization:

$$C \propto \frac{1}{\lambda}.$$

For this first learned-model section, a small log-scale grid is sufficient and more transparent than automated hyperparameter optimization.

6.7 Model comparison

Table 27 reports the cross-validated performance of the main linear classifiers. Logistic regression with L1 and L2 regularization performs almost identically. The L1 model obtains the highest PR-AUC, approximately 0.659, while the L2 model obtains ROC-AUC approximately 0.846 and PR-AUC approximately 0.658. The class-weighted L2 logistic regression has a much higher recall, approximately 0.801, but lower precision, approximately 0.517. This is expected because class weighting makes the model more willing to predict the minority churn class.

The L1 and L2 PR-AUC values differ by only about 0.001. This is much smaller than the kind of difference that should be interpreted as strong statistical evidence. The practical conclusion is that both L1 and L2 logistic regression are competitive in this development-stage comparison. L2 logistic regression is retained as the main coefficient-interpretation model because it is stable, standard, and produces dense coefficients, while L1 remains useful as a sparse alternative.

Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	F1	ROC-AUC	PR-AUC
Logistic L1 C=1	0.802	0.720	0.654	0.543	0.896	0.593	0.846	0.659
Logistic L2 C=1	0.802	0.719	0.652	0.544	0.895	0.593	0.846	0.658
Logistic L2 balanced C=1	0.748	0.765	0.517	0.801	0.729	0.628	0.846	0.657
RidgeClassifier alpha=1	0.801	0.712	0.659	0.521	0.903	0.582	0.838	0.649

Table 27: Cross-validated linear-model comparison on the training set.

The confusion-matrix counts in Table 28 clarify the tradeoff. The standard L2 logistic regression detects 813 churners and misses 682. The class-weighted variant detects 1198 churners but increases false positives from 434 to 1120. Therefore, class weighting improves recall but does so by flagging many more non-churners.

Model	TP	FN	FP	TN
Logistic L1 C=1	812	683	430	3709
Logistic L2 C=1	813	682	434	3705
Logistic L2 balanced C=1	1198	297	1120	3019
RidgeClassifier alpha=1	779	716	403	3736

Table 28: Cross-validated confusion-matrix counts in positive-first order.

6.8 Regularization results

Table 29 and Figure 9 show the L2 regularization path. Very strong regularization, $C = 0.001$, produces a conservative classifier with high precision but low recall. As C increases, recall and F1 improve. The ranking metrics are stable over most of the grid, with ROC-AUC around 0.845 and PR-AUC around 0.658 once $C \geq 0.1$. This indicates that the logistic model is not highly sensitive to the exact L2 regularization strength over the tested range.

This is a good example of why the report should focus on stable hyperparameter regions rather than exact winners. The difference between $C = 0.1$, $C = 1$, $C = 10$, and $C = 100$ is very small for ROC-AUC and PR-AUC. The development-stage evidence mainly says that extremely strong L2 regularization underfits, while a broad range of weaker regularization strengths performs similarly.

C	Acc.	Bal. acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
0.001	0.777	0.599	0.780	0.220	0.343	0.839	0.651
0.01	0.805	0.711	0.674	0.511	0.581	0.843	0.656
0.1	0.801	0.717	0.652	0.538	0.590	0.845	0.658
1	0.802	0.719	0.652	0.544	0.593	0.846	0.658
10	0.804	0.723	0.656	0.549	0.598	0.846	0.658
100	0.804	0.723	0.655	0.550	0.598	0.846	0.657

Table 29: L2 logistic regression regularization grid.

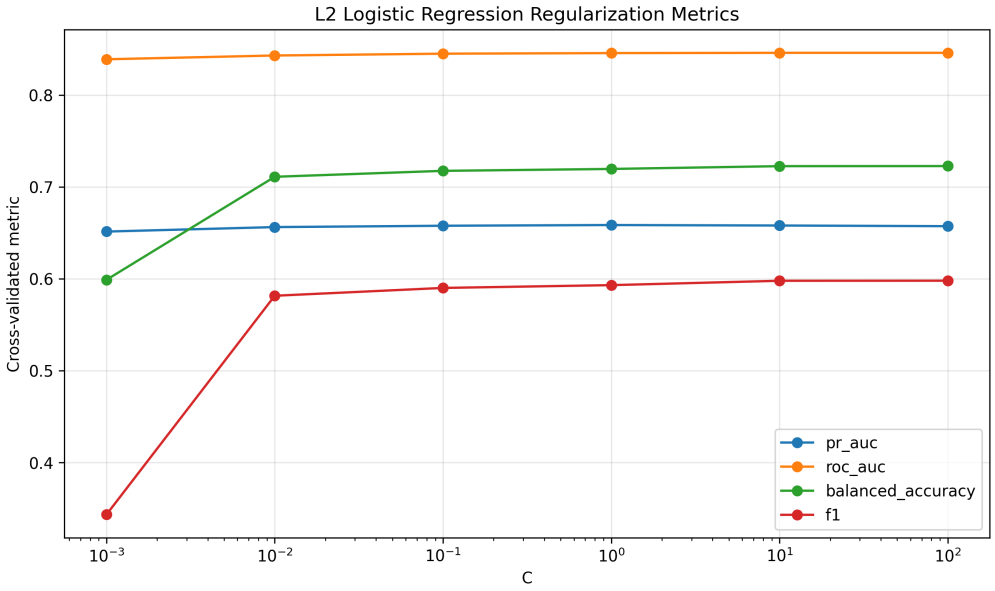

Figure 9: L2 logistic regression metrics over the regularization grid.

The L1 grid in Table 30 and Figure 10 shows a more visible effect of very strong regularization. At $C = 0.001$, the L1 model effectively predicts no churners, so recall and F1 are zero and PR-AUC equals the positive-class prevalence. Once C reaches 0.1 or larger, the L1 model behaves similarly to L2 logistic regression. This suggests that mild L1 regularization is compatible with the predictive structure in the data, but excessive sparsity removes too much useful signal.

Here again, the exact best value should not be overinterpreted. $C = 1$ and $C = 10$ differ only slightly. The important conclusion is that the L1 model needs enough flexibility to keep the relevant one-hot and numeric signals active.

C	Acc.	Bal. acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
0.001	0.735	0.500	0.000	0.000	0.000	0.500	0.265
0.01	0.794	0.673	0.684	0.416	0.517	0.825	0.632
0.1	0.801	0.714	0.655	0.528	0.585	0.844	0.658
1	0.802	0.720	0.654	0.543	0.593	0.846	0.659
10	0.803	0.722	0.654	0.548	0.597	0.846	0.658

Table 30: L1 logistic regression regularization grid.

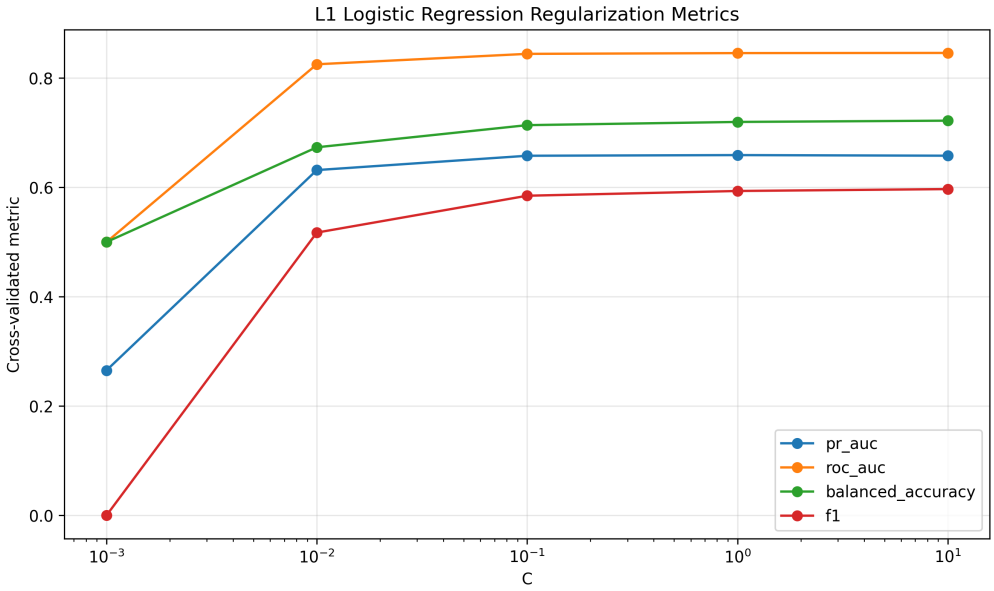

Figure 10: L1 logistic regression metrics over the regularization grid.

6.9 Selected logistic model and coefficient interpretation

For interpretation, the selected model is the L2 logistic regression model with $C = 1$. This is not selected because it is statistically proven to be uniquely optimal. Rather, it is a representative, stable, standard logistic model from the high-performing regularization region. It has development-stage PR-AUC essentially equal to the best L2 configurations, and it supports straightforward coefficient interpretation. The model is fitted on the full training set only for coefficient extraction. These fitted coefficients are not used to report in-sample performance.

Table 31 and Figure 11 show the largest coefficients by absolute value. The strongest negative coefficient is `tenure`, meaning longer-tenure customers receive lower fitted churn scores. `Contract_Two year` also has a negative coefficient, consistent with the EDA finding that two-year contracts have low churn rates. The strongest positive coefficients are `InternetService_Fiber optic`, `Contract_Month-to-month`, and `TotalCharges`, all associated with higher fitted churn scores in this linear model.

A notable pattern is that `MonthlyCharges` has a negative coefficient while `TotalCharges` has a positive coefficient. This should not be interpreted in isolation as a simple marginal effect. Logistic regression estimates partial associations after controlling for the other encoded features. Since `tenure`, `MonthlyCharges`, and `TotalCharges` are related, their coefficients reflect a conditional linear decomposition rather than the marginal patterns seen in EDA.

Feature	Coef.	Odds ratio	Direction
tenure	-1.255	0.285	lower churn score
Contract_Two year	-0.759	0.468	lower churn score
InternetService_Fiber optic	0.648	1.912	higher churn score
InternetService_DSL	-0.644	0.525	lower churn score
MonthlyCharges	-0.601	0.548	lower churn score
Contract_Month-to-month	0.586	1.797	higher churn score
TotalCharges	0.533	1.703	higher churn score
PaperlessBilling_No	-0.331	0.718	lower churn score
DeviceProtection_No internet service	-0.293	0.746	lower churn score
OnlineBackup_No internet service	-0.293	0.746	lower churn score
OnlineSecurity_No internet service	-0.293	0.746	lower churn score
InternetService_No	-0.293	0.746	lower churn score

Table 31: Largest selected logistic regression coefficients by absolute value.

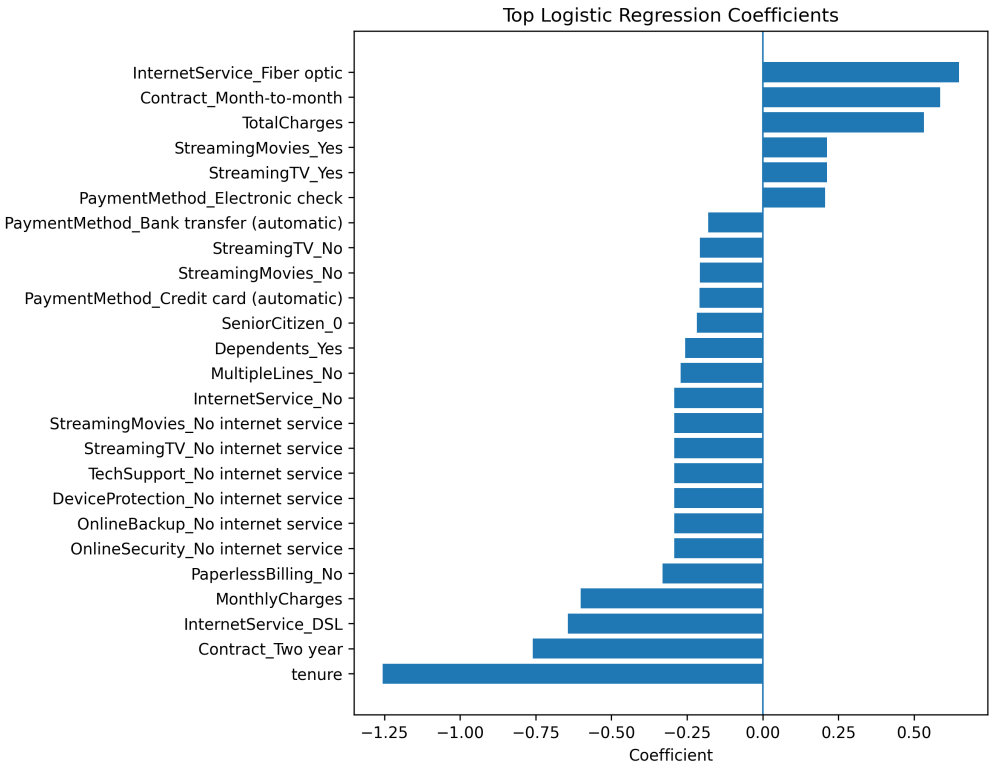

Figure 11: Top logistic regression coefficients by absolute magnitude.

6.10 Threshold tradeoff

The default logistic regression threshold is 0.5, but threshold choice controls the tradeoff between recall, precision, and specificity. Table 32 reports selected thresholds using out-of-fold predicted probabilities. Lowering the threshold increases recall and the predicted positive rate, while raising the threshold increases precision and specificity.

At threshold 0.50, the selected L2 logistic model has precision 0.652, recall 0.544, specificity 0.895, and F1 0.593. At threshold 0.30, recall increases to 0.767, while precision decreases to 0.541 and specificity decreases to 0.765. The F1 score is highest around thresholds 0.30 to 0.35

among the displayed values. This does not mean that the final threshold should be chosen here. Final threshold selection should be postponed until model families are compared and the project objective is specified more explicitly.

Threshold	Acc.	Bal. acc.	Prec.	Recall	Spec.	F1	Pred. pos. rate
0.10	0.611	0.716	0.401	0.942	0.491	0.562	0.624
0.20	0.711	0.757	0.475	0.854	0.659	0.610	0.477
0.25	0.742	0.766	0.509	0.817	0.715	0.627	0.426
0.30	0.766	0.766	0.541	0.767	0.765	0.634	0.376
0.35	0.782	0.761	0.571	0.717	0.805	0.635	0.334
0.40	0.790	0.750	0.594	0.664	0.836	0.627	0.297
0.50	0.802	0.719	0.652	0.544	0.895	0.593	0.221
0.60	0.797	0.669	0.710	0.397	0.942	0.509	0.148
0.70	0.775	0.594	0.779	0.210	0.978	0.331	0.072

Table 32: Threshold tradeoff for selected L2 logistic regression using out-of-fold probabilities.

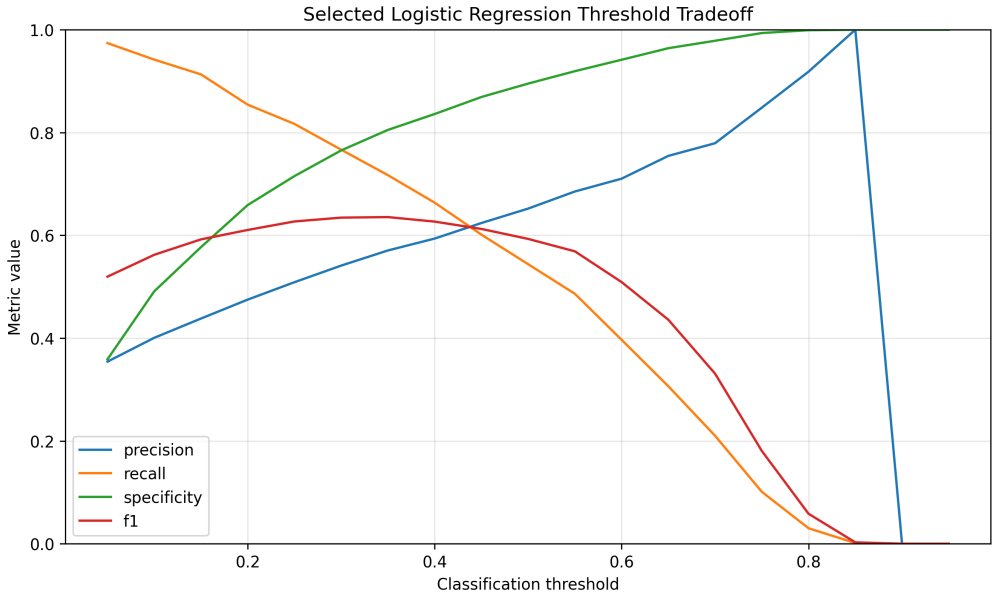

Figure 12: Threshold tradeoff for selected L2 logistic regression.

6.11 ROC and precision-recall curves

Figures 13 and 14 summarize threshold-independent ranking performance for the selected logistic regression model. The ROC curve has AUC approximately 0.846, indicating that the model ranks churners above non-churners substantially better than a random ranking. The precision-recall curve has AUC approximately 0.658, compared with a positive-rate baseline of approximately 0.265. This is important because churn is the minority class, and PR-AUC focuses directly on the quality of positive churn retrieval.

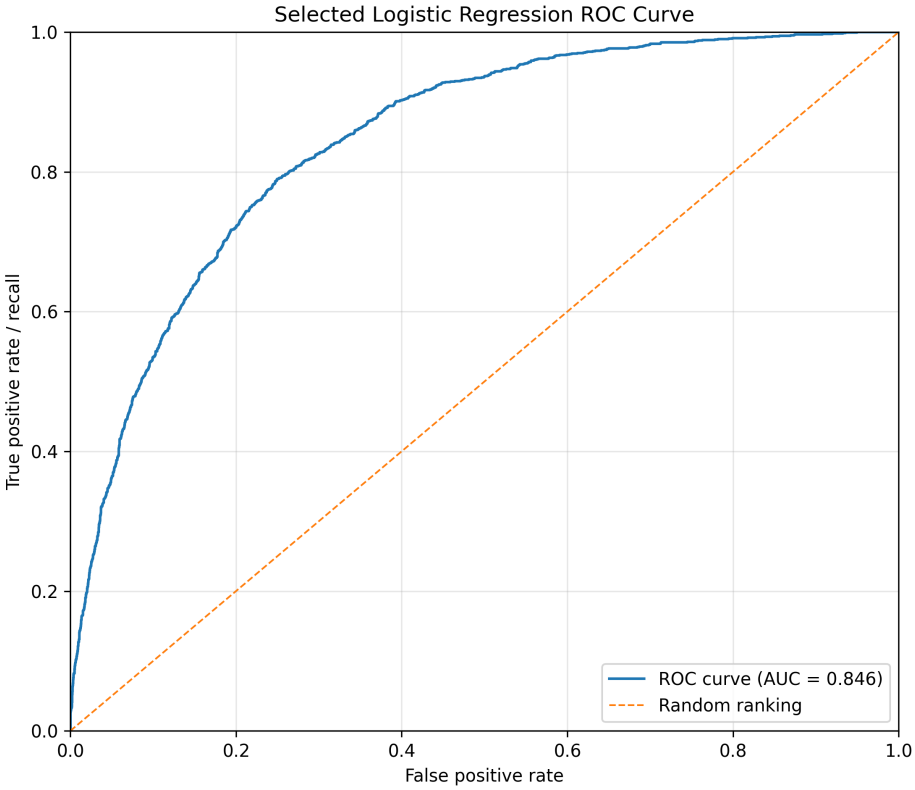

Figure 13: ROC curve for selected L2 logistic regression.

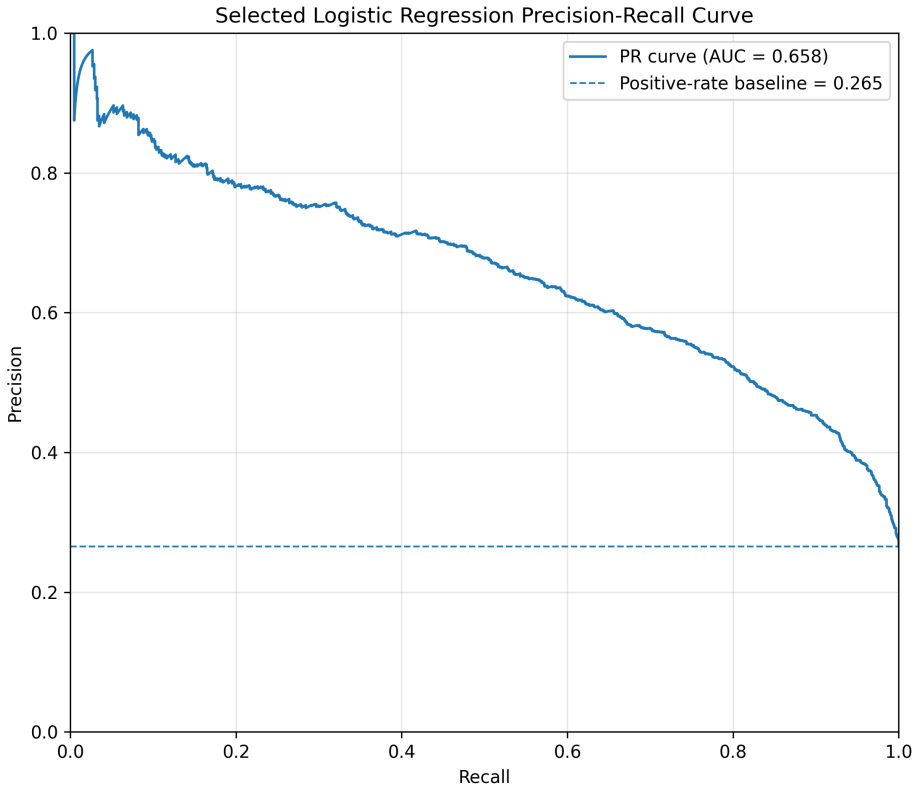

Figure 14: Precision-recall curve for selected L2 logistic regression.

6.12 Summary

Logistic regression is a strong first learned model for the churn task. It clearly improves on non-informative baselines and provides useful probability rankings, with ROC-AUC around 0.846 and PR-AUC around 0.658. L1 and L2 regularization give very similar cross-validated performance once regularization is not excessively strong. Ridge classification is slightly weaker but remains a reasonable least-squares linear baseline.

The regularization grids show that the exact value of C is not the main conclusion. The important development-stage lesson is that extremely strong regularization underfits, while a broad moderate-to-weak regularization region performs similarly. This is why the selected L2 logistic model should be understood as a representative stable linear benchmark rather than a uniquely optimal hyperparameter setting.

The class-weighted logistic regression variant demonstrates the central classification tradeoff: it detects many more churners, but it also creates many more false positives. This confirms that model assessment cannot rely on accuracy alone. Confusion-matrix counts, recall, precision, specificity, ROC-AUC, PR-AUC, and threshold behaviour must be considered together.

The coefficient analysis is also useful. Tenure and two-year contracts are associated with lower fitted churn scores, while fiber optic internet, month-to-month contracts, and higher accumulated charges are associated with higher fitted churn scores. These patterns are consistent with the training-set EDA, but they remain predictive associations rather than causal effects.

This section establishes logistic regression as the first interpretable learned benchmark. Later model families should be compared against it by asking whether they improve ranking performance, reduce false positives at useful recall levels, or capture nonlinear feature interactions that linear models cannot represent directly. Final claims about model-family superiority are deferred to the later comparison and test-evaluation stages.

7 k-Nearest Neighbours

This section evaluates k-nearest neighbours as the first non-parametric and distance-based classifier in the project. Unlike logistic regression, which estimates a global vector of coefficients, k-nearest neighbours stores the training observations and classifies a new customer by comparing that customer with nearby training customers in the transformed feature space. The model is therefore useful as a contrast to the linear classifiers from Section 6: logistic regression learns global feature weights, whereas k-nearest neighbours relies directly on local similarity.

The same validation discipline is used as in the previous modelling sections. The held-out test set is not used. All reported results are based on stratified cross-validation inside the training set. Preprocessing is kept inside the scikit-learn pipeline so that standardization and one-hot encoding are fitted only on each fold's training data.

The results in this section should be interpreted as a development-stage comparison of kNN configurations. The selected kNN model is the strongest configuration in the tried grid according to the chosen selection criterion. It is not a final statistical claim that this exact value of k, distance metric, and weighting rule is uniquely optimal in the population.

7.1 k-nearest-neighbour prediction rule

Let the training set be

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n},$$

where $x_i \in \mathbb{R}^p$ is the transformed feature vector for customer i, and $y_i \in \{0, 1\}$ is the binary churn label. For a new customer with feature vector x, k-nearest neighbours first finds the k training observations closest to x according to a chosen distance function. Let this neighbour set be denoted by

$$\mathcal{N}_k(x).$$

For unweighted binary classification, the estimated churn probability is the fraction of neighbours that churned:

$$\widehat{p}(Y = 1 \mid X = x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i.$$

The default hard classification rule at threshold 0.5 is

$$\widehat{y} = \mathbb{1}\left\{\widehat{p}(Y = 1 \mid X = x) \geq 0.5\right\}.$$

Thus, the model predicts churn when churners form at least half of the local neighbourhood.

A distance-weighted variant gives closer neighbours more influence:

$$\widehat{p}(Y = 1 \mid X = x) = \frac{\sum_{i \in \mathcal{N}_k(x)} w_i(x) y_i}{\sum_{i \in \mathcal{N}_k(x)} w_i(x)}.$$

In this section, both uniform and distance-based voting are evaluated.

7.2 Distance, scaling, and one-hot encoded features

The distance definition is central to k-nearest neighbours. Two common distances are the Manhattan distance,

$$d_1(x, x') = \sum_{j=1}^{p} |x_j - x'_j|,$$

and the Euclidean distance,

$$d_2(x, x') = \sqrt{\sum_{j=1}^{p} (x_j - x'_j)^2}.$$

Both are special cases of the Minkowski distance. In the implementation, Manhattan distance corresponds to $p = 1$, while Euclidean distance corresponds to $p = 2$.

Because k-nearest neighbours computes distances directly from feature values, scaling is essential. Without standardization, variables with larger numeric ranges can dominate the distance calculation. For example, `TotalCharges` has a much larger raw range than `tenure` or `MonthlyCharges`. Therefore, numeric variables are standardized inside the preprocessing pipeline. Categorical variables are one-hot encoded, which allows k-nearest neighbours to operate on mixed tabular data, but also changes the geometry of the feature space. After one-hot encoding, similarity is measured in a higher-dimensional indicator space, which may not always correspond to intuitive customer similarity.

7.3 The role of k

The number of neighbours k controls the smoothness of the classifier. Small values of k produce highly local predictions. For example, when $k = 1$, the prediction copies the label of the single nearest training observation. This can produce low-bias but high-variance decision boundaries that are sensitive to noise.

Larger values of k average over more training observations. This usually increases bias but reduces variance, producing smoother and more stable predictions. If k becomes too large, however, predictions approach the global class distribution and the model can underfit. Therefore, k is a true model-complexity hyperparameter and must be selected using validation data rather than the held-out test set.

7.4 Hyperparameter grid

The k-nearest-neighbour experiment uses a transparent cross-validated grid search. The grid varies the number of neighbours, the voting scheme, and the distance metric:

$$k \in \{1, 3, 5, 7, 11, 15, 21, 31, 51, 75, 101\},$$

$$\texttt{weights} \in \{\texttt{uniform}, \texttt{distance}\},$$

$$p \in \{1, 2\}.$$

The $p = 1$ setting corresponds to Manhattan distance, and $p = 2$ corresponds to Euclidean distance. The selection rule chooses the configuration with the highest cross-validated PR-AUC, using balanced accuracy and F1 as secondary criteria if needed. PR-AUC is used as the primary criterion because churn is the minority class, and the positive-class ranking quality is important.

Because this grid tries multiple configurations, the winning cross-validated score should be interpreted with the model-selection caveat from Section 4. It identifies a strong configuration within the tried grid, but the observed best score can contain some selection optimism.

7.5 Grid-search behaviour

Figure 15 shows the cross-validated PR-AUC across the neighbour grid. Performance improves strongly as k increases from very small values. This pattern indicates that small neighbourhoods

are too noisy for this dataset. Larger neighbourhoods smooth over local irregularities and give more stable churn rankings.

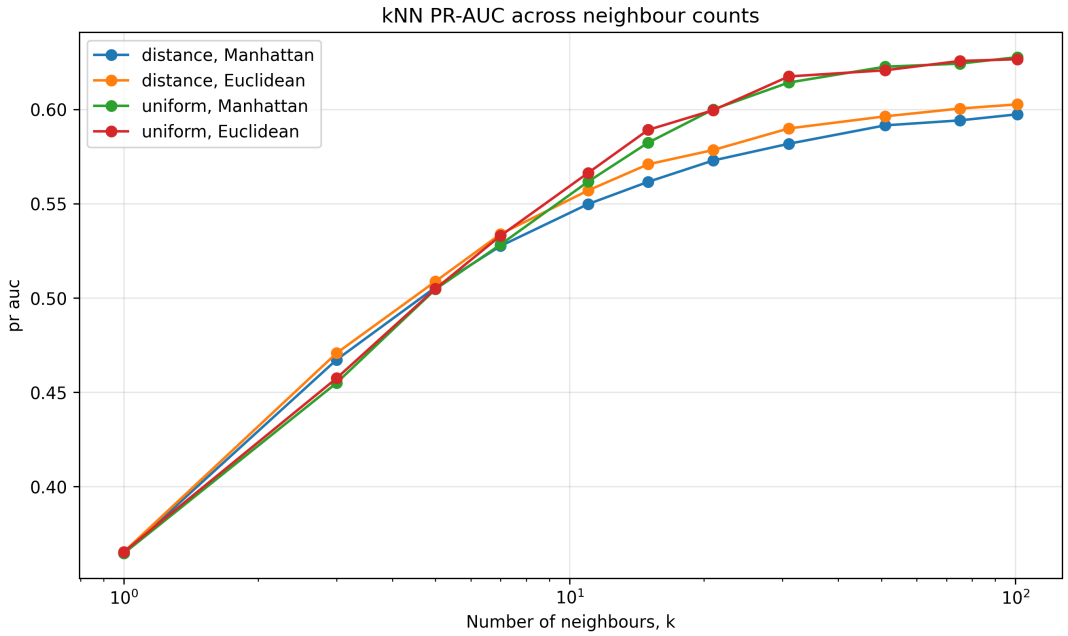

Figure 15: kNN PR-AUC across neighbour counts

Figure 16 shows the same grid from the perspective of balanced accuracy. Balanced accuracy also improves as k increases, but it peaks somewhat earlier than PR-AUC for some configurations. This difference is expected because PR-AUC evaluates ranking quality across thresholds, while balanced accuracy depends on the default hard classification threshold.

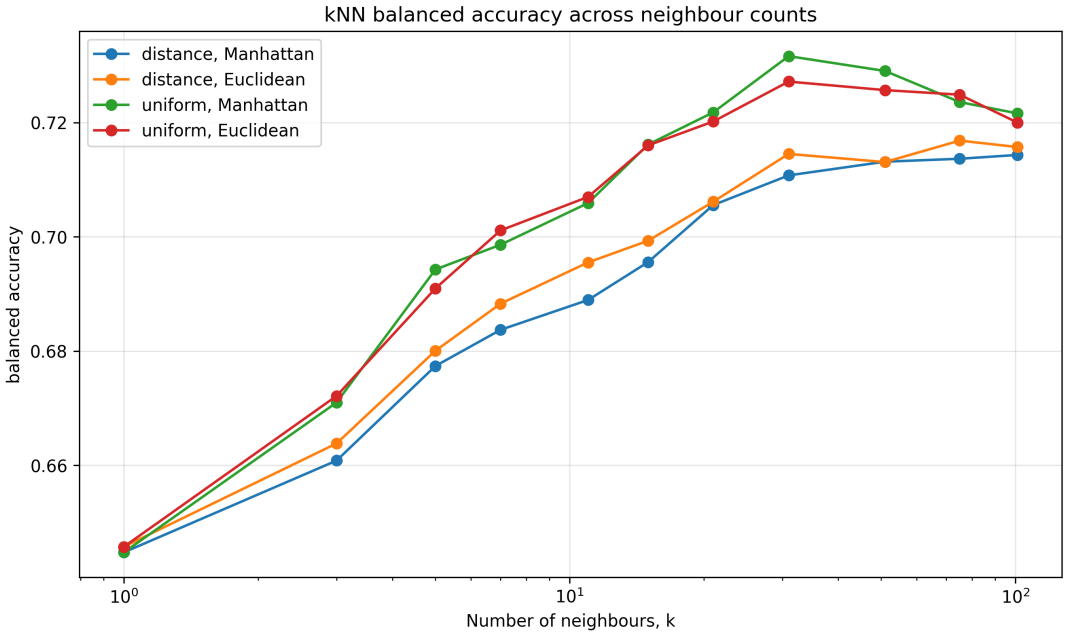

Figure 16: kNN balanced accuracy across neighbour counts

The strongest configuration by PR-AUC in the tried grid is the uniform-weighted Manhattan model with $k = 101$. Uniform weighting outperforms distance weighting among the best configurations.

This suggests that allowing very close neighbours to dominate does not improve development-stage generalization in this transformed feature space. A smoother average over many neighbours is more stable.

The interpretation should focus on the broader pattern rather than the exact integer $k = 101$. The grid evidence suggests that very small neighbourhoods are too noisy and that larger, smoother neighbourhoods work better for this representation. It does not prove that $k = 101$ is uniquely optimal; nearby large-k settings may be practically similar.

7.6 Model comparison

Table 33 compares the default $k = 5$ Euclidean kNN model with the selected kNN model. The selected model improves substantially over the default setting. Accuracy rises from approximately 0.765 to 0.792, balanced accuracy from approximately 0.691 to 0.722, ROC-AUC from approximately 0.782 to 0.836, and PR-AUC from approximately 0.505 to 0.628. This difference is large enough to support the development-stage conclusion that tuning k, distance, and weighting is important for kNN on this dataset.

Model	Accuracy	Bal. Acc.	Precision	Recall	Specificity	F1	ROC-A
Selected kNN, $k = 101$, uniform, Manhattan	0.792	0.722	0.617	0.571	0.872	0.593	0
kNN, $k = 5$, uniform, Euclidean	0.765	0.691	0.561	0.532	0.849	0.546	0

Table 33: kNN model comparison using cross-validated out-of-fold predictions

The corresponding positive-first confusion-matrix counts are shown in Table 34. The selected model detects more churners and also produces fewer false positives than the default $k = 5$ model. Specifically, the selected model gives 854 true positives and 530 false positives, compared with 796 true positives and 623 false positives for the default model.

Model	TP	FN	FP	TN
Selected kNN, $k = 101$, uniform, Manhattan	854	641	530	3609
kNN, $k = 5$, uniform, Euclidean	796	699	623	3516

Table 34: Positive-first confusion-matrix counts for kNN models

Compared with logistic regression from Section 6, selected kNN has lower development-stage ranking metrics. The selected logistic regression model had ROC-AUC around 0.846 and PR-AUC around 0.658, while selected kNN has ROC-AUC around 0.836 and PR-AUC around 0.628. This suggests that the global feature-weighting structure learned by logistic regression is more effective than direct distance-based local similarity for this transformed Telco churn dataset.

This comparison should still be read as development-stage evidence. The gap is meaningful enough to keep logistic regression as the stronger candidate so far, but final statistical claims about model-family superiority are deferred to the later model-comparison and test-evaluation stages.

7.7 Threshold behaviour

The selected kNN model provides predicted probabilities from neighbour proportions. Figure 17 shows how precision, recall, specificity, and F1 change as the classification threshold varies. Lower

thresholds identify many more churners but also flag many more non-churners. Higher thresholds improve precision and specificity but miss more churners.

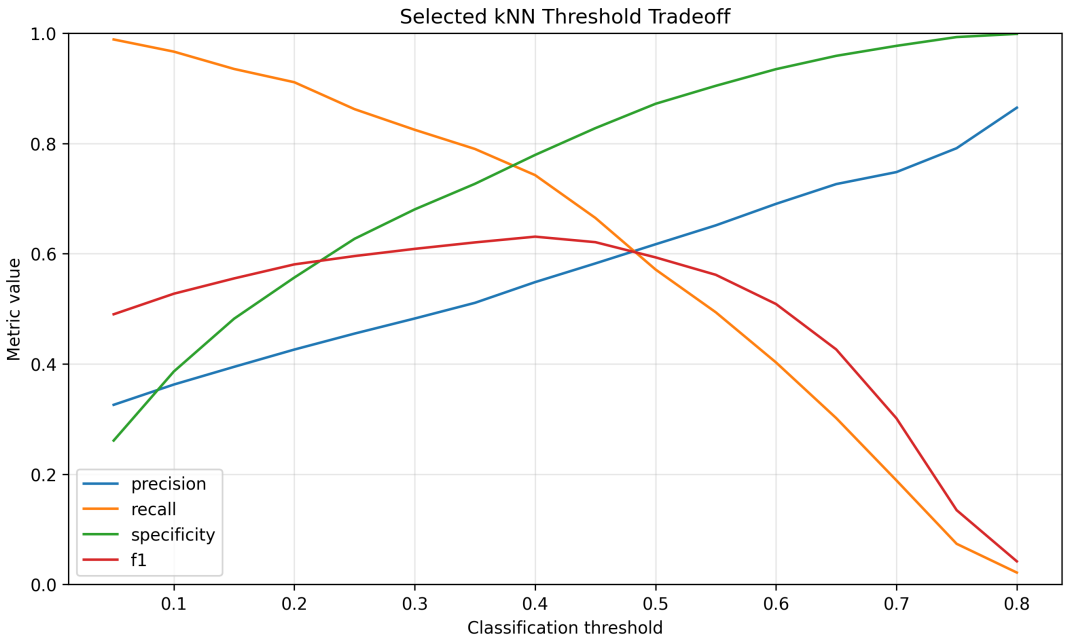

Figure 17: Threshold tradeoff for the selected kNN model

At threshold 0.25, recall is approximately 0.862, but precision is only approximately 0.455. At threshold 0.40, the model reaches a more balanced operating point, with precision approximately 0.548, recall approximately 0.742, specificity approximately 0.779, and F1 approximately 0.631. At the default threshold 0.50, precision improves to approximately 0.617, but recall falls to approximately 0.571. This again shows that threshold 0.50 is not necessarily the best operating point for churn detection.

These threshold results are diagnostic rather than final. Threshold selection is itself a modelling decision and is postponed until the later model-comparison stage, where the project objective and candidate model families can be considered together.

7.8 ROC and precision-recall curves

Figure 18 shows the ROC curve for the selected kNN model. The ROC-AUC is approximately 0.836, clearly above the random-ranking line. This indicates that the selected kNN model ranks churners above non-churners much better than chance.

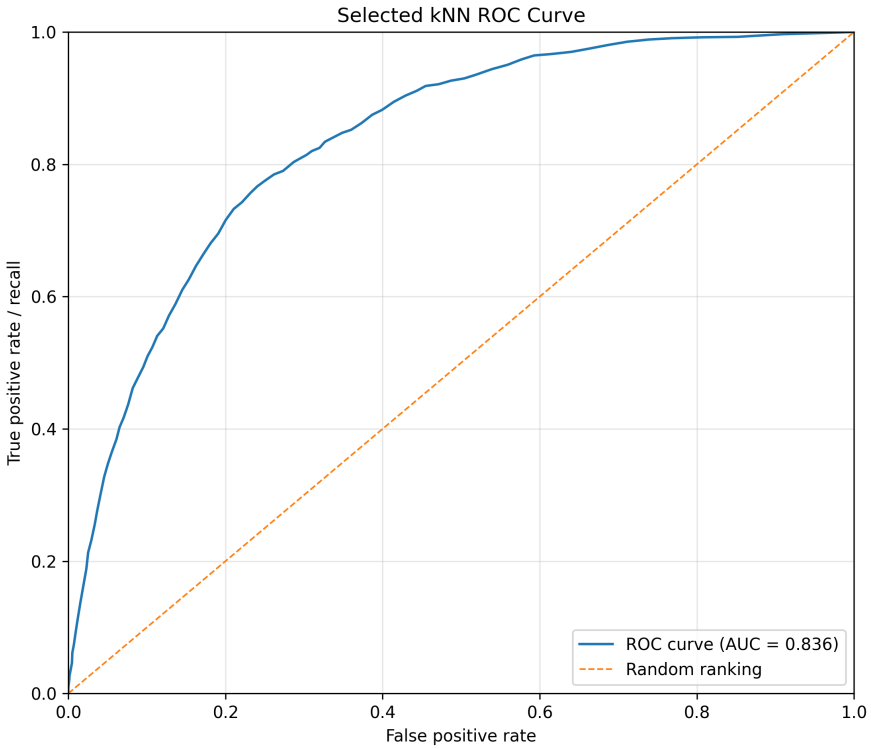

Figure 18: ROC curve for the selected kNN model

Figure 19 shows the precision-recall curve. The PR-AUC is approximately 0.631, well above the positive-rate baseline of approximately 0.265. This confirms that kNN has useful positive-class retrieval ability. However, it remains below the logistic regression PR-AUC from Section 6. The gap is not extremely large, so it should be interpreted as development-stage evidence rather than final statistical proof.

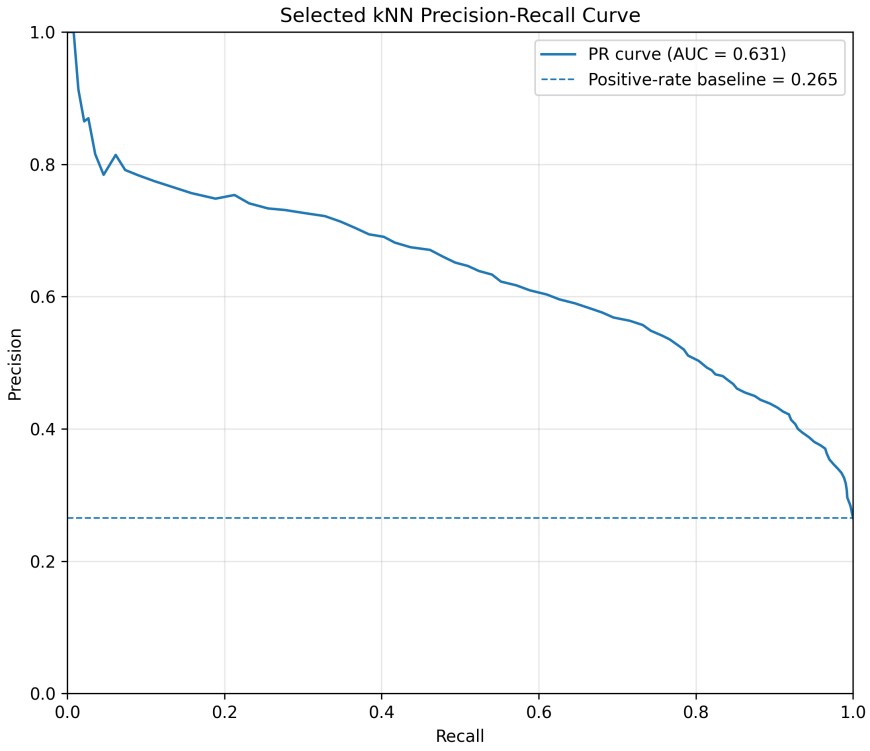

Figure 19: Precision-recall curve for the selected kNN model

7.9 Summary

The k-nearest-neighbour experiment shows that distance-based classification learns meaningful churn structure, but it does not outperform logistic regression in the current development-stage comparison. The most important findings are as follows. First, kNN performance is highly sensitive to the number of neighbours. Small k values are too noisy, while larger neighbourhoods perform better. Second, uniform weighting works better than distance weighting in the strongest configurations. Third, the strongest PR-AUC in the grid is obtained by the uniform Manhattan model with $k = 101$, which is a relatively smooth kNN classifier. Fourth, the selected kNN model improves substantially over the default $k = 5$ baseline, but remains below logistic regression in ROC-AUC and PR-AUC.

The section is useful because it introduces local, distance-based learning and the bias-variance role of k. It also illustrates a limitation of distance-based methods on one-hot encoded tabular data: the constructed distance space may not be as informative as a model that learns feature weights directly. The exact selected k should not be overinterpreted; the robust lesson is that smoothing over a larger neighbourhood is preferable to very local neighbour voting for this feature representation. The next modelling stage introduces Naive Bayes before the project moves to tree-based models.

8 Naive Bayes

This section evaluates Naive Bayes classifiers as the first explicitly generative model family in the project. Logistic regression directly models the posterior probability $P(Y = 1 \mid X = x)$, and k-nearest neighbours predicts from local similarity in the transformed feature space. Naive Bayes instead models the class prior and the feature distribution within each class, then applies Bayes' rule to obtain posterior class probabilities.

The same training-only validation discipline is used as in the previous modelling sections. The held-out test set remains unused. All reported results are based on stratified cross-validation inside the training set, using out-of-fold predictions.

The comparisons in this section are development-stage comparisons of Naive Bayes variants. The selected variant is the strongest Naive Bayes candidate in the tried set according to the chosen metric and modelling rationale. This does not constitute a final statistical superiority claim over other model families.

8.1 Bayes classifier

For binary classification, the ideal classifier predicts the class with the largest true posterior probability:

$$h^\star(x) = \arg\max_{y \in \{0,1\}} P(Y = y \mid X = x).$$

For churn classification, where $Y = 1$ denotes churn and $Y = 0$ denotes no churn, this can be written as

$$h^\star(x) = \mathbb{1}\{P(Y = 1 \mid X = x) \geq 0.5\}.$$

This rule is called the Bayes classifier. It is ideal because it uses the true conditional probability of the class given the features. In practice, this posterior probability is unknown and must be estimated from data.

Under 0-1 loss, the risk of a classifier h is

$$R(h) = P(h(X) \neq Y) = \mathbb{E}\left[\mathbb{1}\{h(X) \neq Y\}\right].$$

The Bayes classifier minimizes this risk over all possible classifiers:

$$h^\star = \arg\min_h R(h).$$

The corresponding minimum possible error rate is the Bayes risk,

$$R^\star = R(h^\star).$$

For binary classification, the Bayes risk can be written as

$$R^\star = \mathbb{E}_X\left[\min\{P(Y = 0 \mid X), P(Y = 1 \mid X)\}\right].$$

This expression shows that classification error can be irreducible. If churners and non-churners overlap for the same observed feature values, then even the ideal classifier must sometimes be wrong.

8.2 Bayes decision rule and unequal costs

The threshold 0.5 is optimal only when false positives and false negatives have equal cost. In churn prediction, this is not necessarily realistic. A false positive may cause unnecessary retention effort, while a false negative may mean missing a customer who will leave.

Let C_{FP} denote the cost of a false positive and C_{FN} the cost of a false negative. A cost-sensitive Bayes decision rule predicts churn when

$$C_{\mathrm{FP}} P(Y = 0 \mid X = x) \leq C_{\mathrm{FN}} P(Y = 1 \mid X = x).$$

Equivalently, predict churn when

$$P(Y = 1 \mid X = x) \geq \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}.$$

If missing a churner is more costly than unnecessarily contacting a non-churner, then $C_{\mathrm{FN}} > C_{\mathrm{FP}}$, and the optimal threshold is below 0.5. This provides the theoretical reason why the project studies threshold tradeoffs for probabilistic classifiers.

8.3 Bayes' rule and generative classification

Bayes' rule gives

$$P(Y = y \mid X = x) = \frac{P(X = x \mid Y = y) P(Y = y)}{P(X = x)}.$$

For classification, the denominator $P(X = x)$ is the same for every class. Therefore, the predicted class can be obtained by comparing the unnormalized posterior scores:

$$\widehat{y} = \arg\max_{y \in \{0,1\}} P(X = x \mid Y = y) P(Y = y).$$

The term $P(Y = y)$ is the class prior. The term $P(X = x \mid Y = y)$ is the class-conditional feature likelihood.

Naive Bayes is a generative classifier because it models $P(Y)$ and $P(X \mid Y)$. Conceptually, the model describes a process in which a class is first drawn from the class prior, and features are then drawn from the class-conditional feature distribution.

8.4 The naive conditional-independence assumption

The full class-conditional feature distribution is

$$P(X = x \mid Y = y) = P(X_1 = x_1, \ldots, X_p = x_p \mid Y = y).$$

Estimating this full joint distribution is difficult when the feature vector is high-dimensional. Naive Bayes makes the simplifying assumption that features are conditionally independent given the class:

$$P(X = x \mid Y = y) = \prod_{j=1}^{p} P(X_j = x_j \mid Y = y).$$

The posterior score is therefore proportional to

$$P(Y = y) \prod_{j=1}^{p} P(X_j = x_j \mid Y = y).$$

In practice, computations are done on the log scale:

$$\log P(Y = y) + \sum_{j=1}^{p} \log P(X_j = x_j \mid Y = y).$$

This avoids numerical underflow and turns a product of probabilities into a sum of feature-level log-likelihood contributions.

The independence assumption is not literally true for this dataset. For example, `tenure` and `TotalCharges` are strongly related, and the internet-service variables are structurally connected to service add-ons such as `OnlineSecurity`, `OnlineBackup`, and `TechSupport`. Therefore, Naive Bayes may double-count related evidence and produce overconfident probabilities. Nevertheless, it can still be useful as a classifier if the posterior ranking remains informative.

8.5 Naive Bayes variants

This section evaluates four transparent Naive Bayes variants.

First, Gaussian Naive Bayes is applied to the numeric features only. Gaussian Naive Bayes assumes

$$X_j \mid Y = y \sim \mathcal{N}(\mu_{jy}, \sigma_{jy}^2),$$

with class-specific means and variances. This model uses only `tenure`, `MonthlyCharges`, and `TotalCharges`.

Second, Bernoulli Naive Bayes is applied to one-hot encoded categorical features only. For a binary indicator $X_j \in \{0, 1\}$, Bernoulli Naive Bayes uses

$$P(X_j = x_j \mid Y = y) = \theta_{jy}^{x_j}(1 - \theta_{jy})^{1-x_j}.$$

Additive smoothing is used to avoid zero probabilities. With smoothing parameter α, estimated probabilities are pulled away from exactly zero and one.

Third, a hybrid Gaussian-Bernoulli Naive Bayes model is applied to the full mixed feature representation. This model uses Gaussian likelihoods for the numeric features and Bernoulli likelihoods for the one-hot categorical indicators. If $\mathcal{N}$ denotes the numeric feature block and $\mathcal{B}$ denotes the binary one-hot block, the hybrid likelihood is

$$P(X = x, Z = z \mid Y = y) = \prod_{j \in \mathcal{N}} p(x_j \mid Y = y) \prod_{k \in \mathcal{B}} P(z_k \mid Y = y).$$

The corresponding class log score is

$$\log P(Y = y) + \sum_{j \in \mathcal{N}} \log p(x_j \mid Y = y) + \sum_{k \in \mathcal{B}} \log P(z_k \mid Y = y).$$

This is the most theoretically natural Naive Bayes specification for the mixed Telco feature space.

Fourth, Gaussian Naive Bayes is applied to the full transformed feature matrix, including numeric variables and one-hot encoded categorical indicators. This is not the cleanest theoretical likelihood for binary indicators, but it provides a useful comparison with the hybrid model.

8.6 Model comparison

Table 35 reports the cross-validated results for the four main Naive Bayes variants. The selected model by PR-AUC is the hybrid Gaussian-Bernoulli Naive Bayes model. It has PR-AUC approximately 0.615, ROC-AUC approximately 0.822, and recall approximately 0.809. This result is

important because the selected model is both the strongest Naive Bayes model by the chosen ranking metric and the most theoretically coherent likelihood specification for the mixed feature space.

The performance gap between the hybrid model and the full transformed GaussianNB model is useful development evidence, but it should not be overstated as final statistical proof. The improvement in PR-AUC is modest, from approximately 0.605 to 0.615. The stronger reason for preferring the hybrid specification in this section is the combination of a better development-stage PR-AUC and a more appropriate likelihood structure for mixed numeric and binary features.

Model	Accuracy	Bal. Acc.	Precision	Recall	Specificity	F_1	ROC-AUC	PR-AUC
Hybrid Gaussian-BernoulliNB, $\alpha = 1$	0.727	0.753	0.491	0.809	0.697	0.611	0.822	0.615
GaussianNB full transformed	0.698	0.746	0.462	0.847	0.644	0.598	0.819	0.605
BernoulliNB categorical only, $\alpha = 1$	0.721	0.751	0.485	0.814	0.687	0.608	0.815	0.596
GaussianNB numeric only	0.764	0.696	0.557	0.549	0.842	0.553	0.774	0.584

Table 35: Cross-validated Naive Bayes model comparison on the training set

The corresponding confusion-matrix counts are shown in Table 36. The selected hybrid model detects 1209 of the 1495 churners and misses 286. It also incorrectly flags 1253 non-churners. Compared with the full transformed GaussianNB model, the hybrid model gives up some recall but removes substantially more false positives. This is why it has better precision, F_1, and PR-AUC.

Model	TP	FN	FP	TN
Hybrid Gaussian-BernoulliNB, $\alpha = 1$	1209	286	1253	2886
GaussianNB full transformed	1267	228	1475	2664
BernoulliNB categorical only, $\alpha = 1$	1217	278	1294	2845
GaussianNB numeric only	821	674	653	3486

Table 36: Positive-first confusion-matrix counts for Naive Bayes models

The categorical-only Bernoulli model remains close to the hybrid model, showing that categorical features contain much of the Naive Bayes signal. The numeric-only Gaussian model has higher ordinary accuracy, but lower recall and weaker ranking performance. This confirms that ordinary accuracy is again not sufficient for comparing churn classifiers.

Compared with logistic regression and k-nearest neighbours, the selected hybrid Naive Bayes model is still weaker in ranking metrics. Logistic regression had ROC-AUC around 0.846 and PR-AUC around 0.658, while selected kNN had ROC-AUC around 0.836 and PR-AUC around 0.628. The hybrid Naive Bayes model has ROC-AUC around 0.822 and PR-AUC around 0.615. It is useful and theoretically coherent, but it is not the strongest model family so far under the development-stage ranking metrics. Final model-family claims are deferred to the later comparison and test-evaluation stages.

8.7 Smoothing sensitivity for Bernoulli Naive Bayes

Figure 20 shows the Bernoulli Naive Bayes smoothing experiment. The metrics are almost flat across the tested range of α. This means that additive smoothing is not a major source of performance variation for this dataset and feature representation.

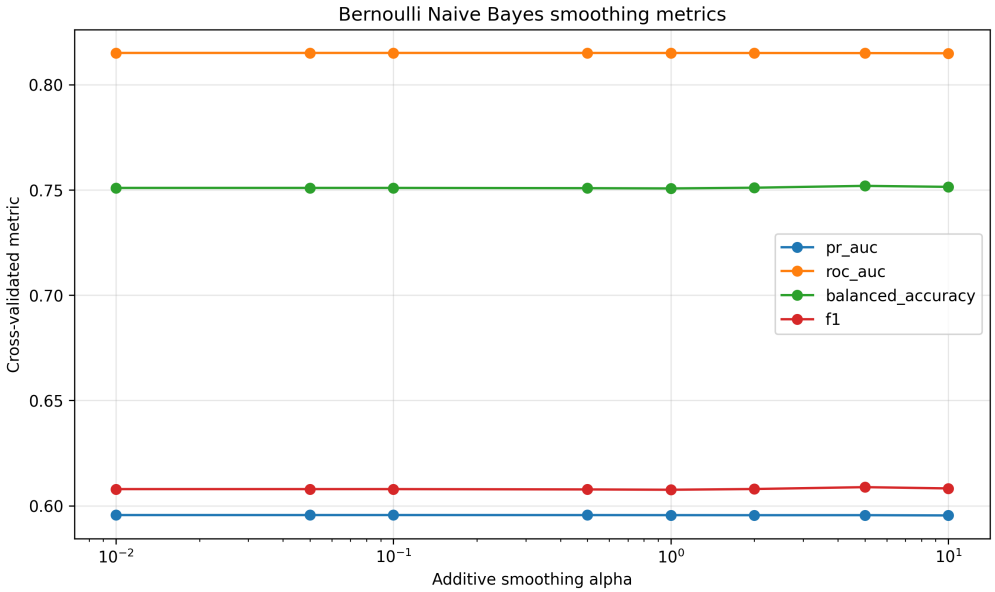

Figure 20: Bernoulli Naive Bayes smoothing metrics

For example, PR-AUC remains around 0.596 across the grid, ROC-AUC remains around 0.815, and balanced accuracy remains close to 0.751. This stability suggests that the categorical indicator likelihoods are not overly sensitive to rare zero-probability problems under this cross-validation setup. It also means that the exact smoothing value should not be overinterpreted; the useful conclusion is that BernoulliNB performance is stable over the tested smoothing range.

8.8 Threshold behaviour

Figure 21 shows the threshold tradeoff for the selected hybrid Naive Bayes model. The curve remains relatively flat compared with logistic regression and kNN, but it is less extreme than the earlier full GaussianNB variant. Increasing the threshold improves precision and specificity while recall declines gradually.

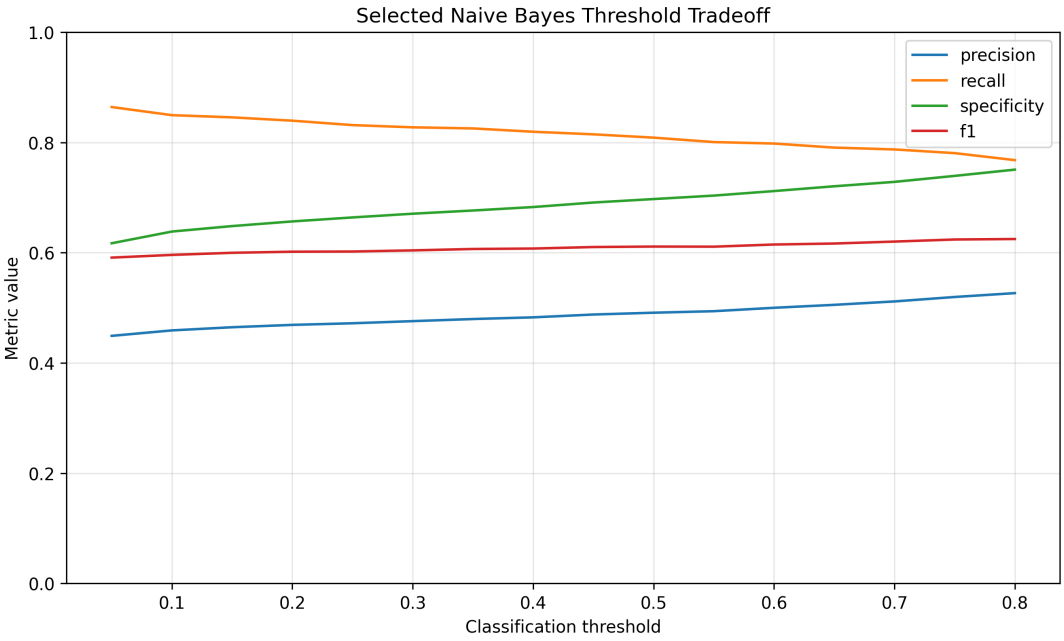

Figure 21: Threshold tradeoff for the selected hybrid Naive Bayes model

At threshold 0.50, the selected model has precision approximately 0.491, recall approximately 0.809, specificity approximately 0.697, and F_1 approximately 0.611. At threshold 0.80, precision rises to approximately 0.527, recall remains high at approximately 0.768, specificity rises to approximately 0.751, and F_1 is approximately 0.625. This illustrates that the default threshold is not necessarily the best operating point. However, final threshold selection is postponed until later model comparison.

The broad flatness of the threshold curve is still consistent with the Naive Bayes independence assumption being imperfect for correlated Telco features. The hybrid model improves the feature-type likelihood specification, but it still assumes conditional independence within the numeric and binary feature blocks.

8.9 ROC and precision-recall curves

Figure 22 shows the ROC curve for the selected hybrid Naive Bayes model. The ROC-AUC is approximately 0.822, clearly above the random-ranking line. This confirms that the hybrid model learns meaningful churn-discriminating structure.

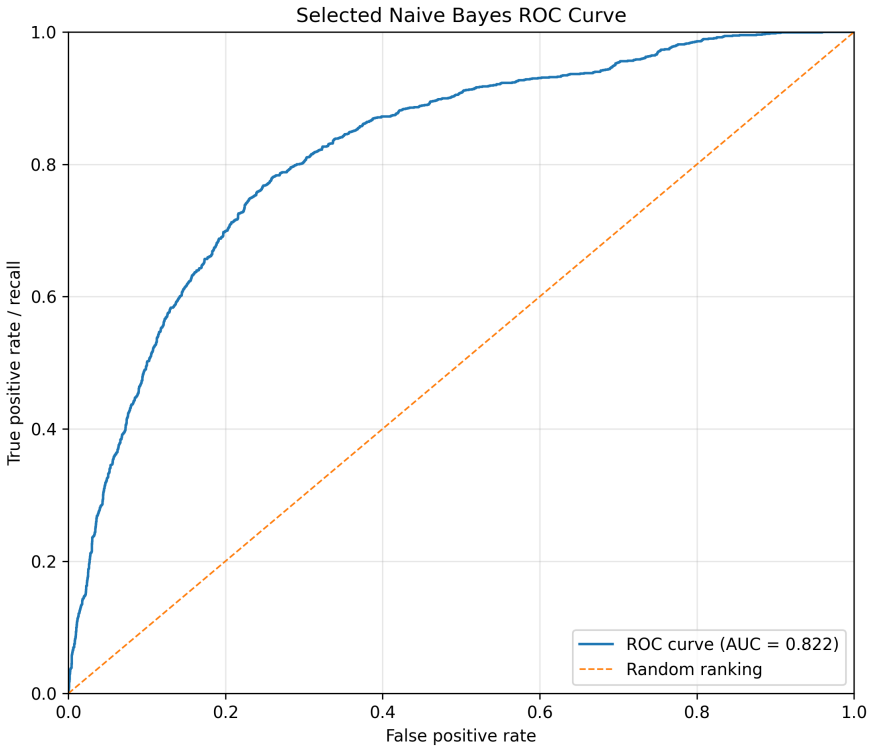

Figure 22: ROC curve for the selected hybrid Naive Bayes model

Figure 23 shows the precision-recall curve. The PR-AUC is approximately 0.614, well above the positive-rate baseline of approximately 0.265. This indicates useful positive-class retrieval ability. However, the PR-AUC remains lower than the corresponding logistic regression and kNN values, so Naive Bayes is not the strongest ranking model so far in the development-stage comparison.

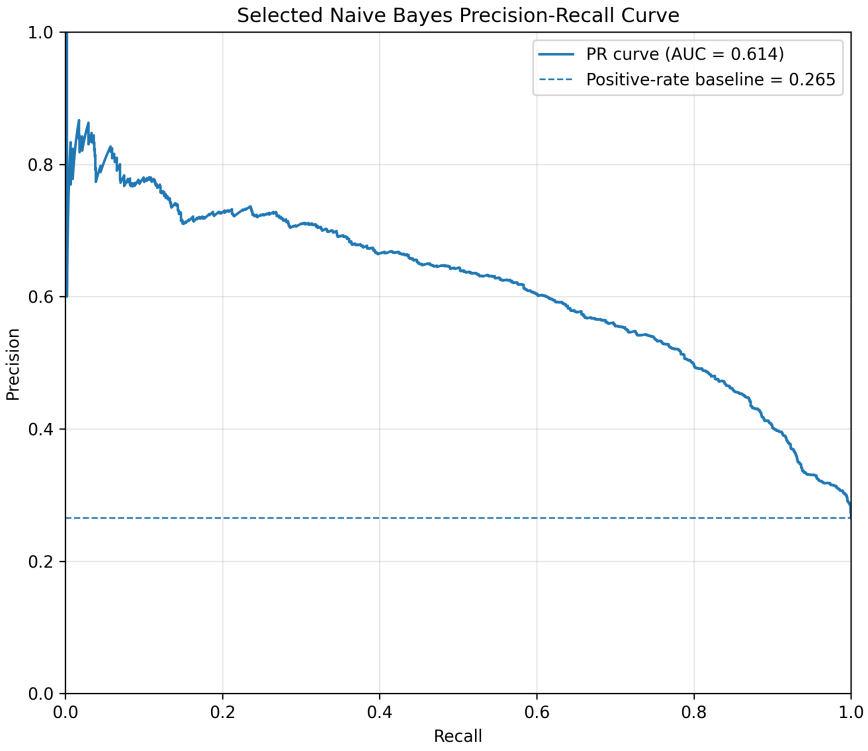

Figure 23: Precision-recall curve for the selected hybrid Naive Bayes model

8.10 Summary

The revised Naive Bayes experiment shows that a simple generative model can capture meaningful churn patterns. Adding the hybrid Gaussian-Bernoulli specification improves the section because it uses a likelihood structure that matches the mixed feature representation: Gaussian likelihoods for continuous numeric variables and Bernoulli likelihoods for one-hot categorical indicators.

The selected hybrid model has high recall, moderate precision, and useful ranking performance. It improves over the full transformed GaussianNB model, which treated binary indicators as Gaussian variables. The observed improvement is modest: PR-AUC rises from about 0.605 to about 0.615, and the number of false positives falls from 1475 to 1253. Therefore, the result should be interpreted as a practical and theoretically motivated Naive Bayes selection, not as a final statistical superiority claim.

The main limitation remains the conditional-independence assumption. Many Telco features are correlated, especially service-related one-hot indicators. Therefore, Naive Bayes probabilities should be interpreted carefully and should not be treated as calibrated churn probabilities without later calibration checks.

Compared with the learned models evaluated so far, Naive Bayes is useful but not dominant. Logistic regression remains the strongest model family so far by development-stage PR-AUC and ROC-AUC, while kNN also remains stronger than Naive Bayes on these ranking metrics. Nevertheless, Naive Bayes adds an important modelling perspective: it connects classification to Bayes' rule, Bayes optimality, class priors, class-conditional likelihoods, smoothing, and the practical difference between ideal Bayes classification and approximate generative modelling.

9 Decision Trees

This section evaluates single decision-tree classifiers for churn prediction. Decision trees are useful at this stage because they introduce nonlinear, rule-based classification without yet using ensembles. A fitted tree partitions the transformed feature space into regions, assigns each region a local churn proportion, and uses that local proportion both for hard classification and for ranking customers by churn risk.

The same development discipline is used as in the previous modelling sections. The held-out test set is not used. All reported results are based on stratified cross-validation inside the training set, using out-of-fold predictions where possible. The comparisons in this section are therefore development-stage comparisons of decision-tree variants. They identify a representative single-tree candidate within the tried configurations, but they are not final test-set performance claims.

9.1 Recursive partitioning

Let the training data be

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n},$$

where x_i is the transformed customer feature vector and $y_i \in \{0, 1\}$ is the churn label. A decision tree recursively splits the feature space into smaller regions. Each internal node applies a rule of the form

$$x_j \leq s,$$

where x_j is one feature and s is a split value. For one-hot encoded categorical indicators, these split rules often test whether a category indicator is zero or one.

After the recursive splitting process, each terminal node or leaf contains a subset of training observations. For a leaf R_m, the estimated churn probability is the local class proportion:

$$\widehat{p}_m = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} y_i.$$

For a new customer x, the fitted tree sends x to one leaf and predicts

$$\widehat{p}(Y = 1 \mid X = x) = \widehat{p}_m.$$

The default hard prediction at threshold 0.5 is

$$\widehat{y} = \mathbb{1}\{\widehat{p}(Y = 1 \mid X = x) \geq 0.5\}.$$

Thus, a classification tree can be interpreted as a collection of local empirical churn-rate rules.

9.2 Split selection and impurity

Tree construction is greedy. At a node, the algorithm evaluates candidate feature-threshold splits and chooses the split that most improves class purity. It does not search globally over all possible trees, because the number of possible tree structures is too large.

For binary classification, one common impurity measure is the Gini impurity. If a node has churn proportion p, its Gini impurity is

$$G(p) = 1 - p^2 - (1 - p)^2 = 2p(1 - p).$$

The impurity is zero when the node is pure, meaning $p = 0$ or $p = 1$. It is largest when the node is evenly mixed, meaning $p = 0.5$.

For a candidate split that divides a parent node R into left and right children R_L and R_R, the weighted child impurity is

$$G_{\text{children}} = \frac{|R_L|}{|R|} G(R_L) + \frac{|R_R|}{|R|} G(R_R).$$

The split improvement is

$$\Delta G = G(R) - G_{\text{children}}.$$

The greedy algorithm chooses the candidate split with the largest impurity reduction. Entropy and information gain follow the same logic, but use

$$H(p) = -p \log(p) - (1 - p) \log(1 - p)$$

instead of Gini impurity.

9.3 Ranking with decision trees

Although decision trees are often described as rule-based classifiers, they also produce scores. The score for a customer is the estimated churn proportion in the leaf that the customer reaches. Therefore, decision trees rank customers by leaf churn rate.

This ranking has a specific structure. All customers in the same leaf receive exactly the same score, so the ranking is stepwise and can contain many ties. A shallow or strongly pruned tree has fewer leaves and therefore fewer possible score levels. This makes its ranking coarse but often more stable. A deeper tree has more leaves and more possible score levels, but the estimated leaf probabilities can be noisy when leaves contain few observations.

This matters for ROC-AUC and precision-recall analysis. Trees are trained by local impurity reduction, not by directly optimizing ROC-AUC or PR-AUC. Ranking quality must therefore be evaluated explicitly from out-of-fold predicted probabilities.

9.4 Regularization and pruning

Unrestricted decision trees can overfit. A deep tree can keep splitting until leaves become very small, fitting local noise and idiosyncratic training patterns. This lowers training impurity but can harm validation performance.

This section evaluates two forms of tree regularization. The first is pre-pruning, where tree growth is restricted during fitting through hyperparameters such as

`max_depth`, `min_samples_split`, `min_samples_leaf`.

The second is cost-complexity pruning, controlled in scikit-learn by `ccp_alpha`. Cost-complexity pruning grows a large tree and then selects a smaller subtree by penalizing complexity. Conceptually, the pruning criterion has the form

$$\text{training impurity} + \alpha \times \text{tree complexity},$$

where larger α favours smaller trees.

The pruning parameter is a hyperparameter like `max_depth` or `min_samples_leaf`. In this section it is selected with cross-validation inside the training set. This is appropriate for development-stage

model selection. However, if a separate validation set were reserved for higher-level model-family comparison, that same validation set should not also be used to choose the pruning level. A stricter estimate of the full tune-and-select procedure would require nested validation. This section does not claim to provide such a final unbiased estimate; final testing remains deferred.

9.5 Experimental design

Four decision-tree variants are evaluated:

- **Decision stump**: a depth-one tree.
- **Default decision tree**: an unrestricted baseline tree.
- **Pre-pruned tree grid**: a grid over `criterion`, `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- **Cost-complexity-pruned tree grid**: a grid over candidate `ccp_alpha` values.

The representative decision tree is selected by cross-validated PR-AUC, with balanced accuracy and F_1 used as secondary criteria. PR-AUC remains the primary criterion because churn is the minority class and positive-class retrieval is central to the project.

9.6 Pre-pruning results

Figure 24 shows the pre-pruning grid from the perspective of PR-AUC. Very shallow trees underfit the ranking problem, while moderately deep trees perform better. The best PR-AUC occurs at depth 6 with `min_samples_leaf` = 10. The unrestricted-depth setting performs worse, especially when `min_samples_leaf` = 1, which is consistent with overfitting.

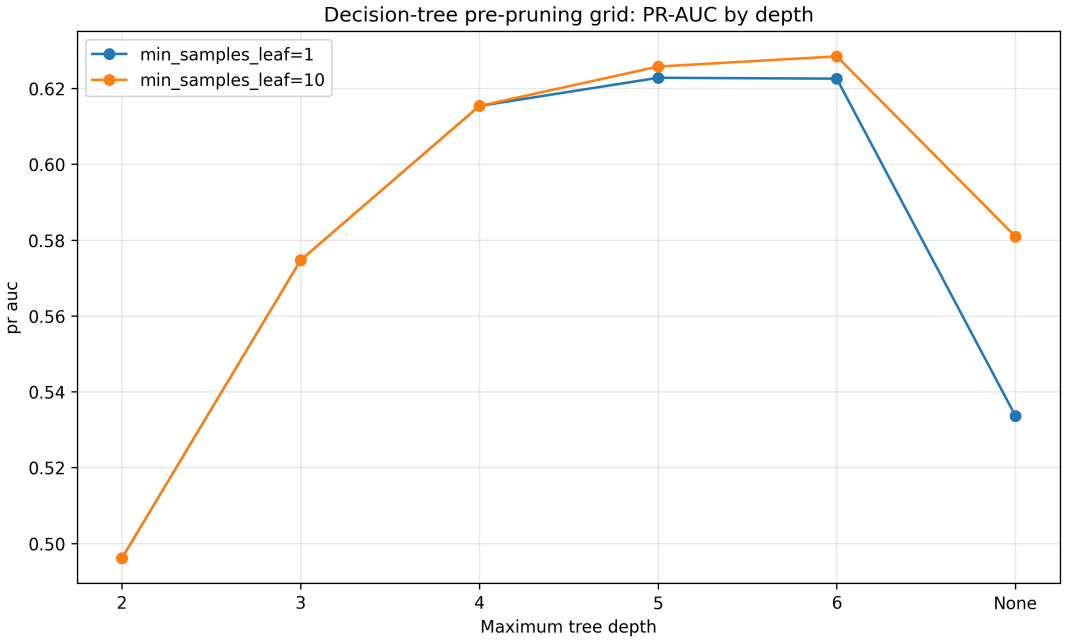

Figure 24: Decision-tree pre-pruning grid: PR-AUC by maximum depth

Figure 25 shows the same grid using balanced accuracy. The depth-two tree has high balanced accuracy but weak PR-AUC, showing that a good default-threshold class split is not the same

as strong ranking performance. The depth-five and depth-six trees provide stronger ranking performance while keeping the tree sufficiently regularized.

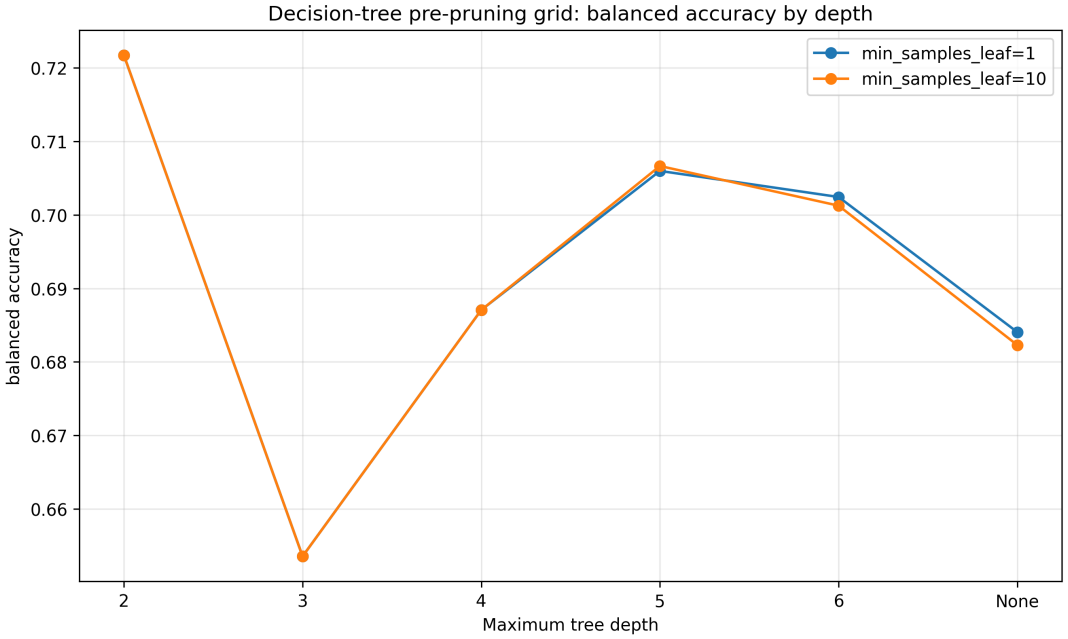

Figure 25: Decision-tree pre-pruning grid: balanced accuracy by maximum depth

9.7 Cost-complexity pruning results

Figure 26 shows the cost-complexity pruning grid. The unpruned tree has weak PR-AUC because it overfits. As `ccp_alpha` increases, the tree becomes smaller and the ranking metrics improve substantially. The best cost-complexity-pruned tree reaches PR-AUC approximately 0.615, which is much better than the default unrestricted tree but still below the best pre-pruned tree.

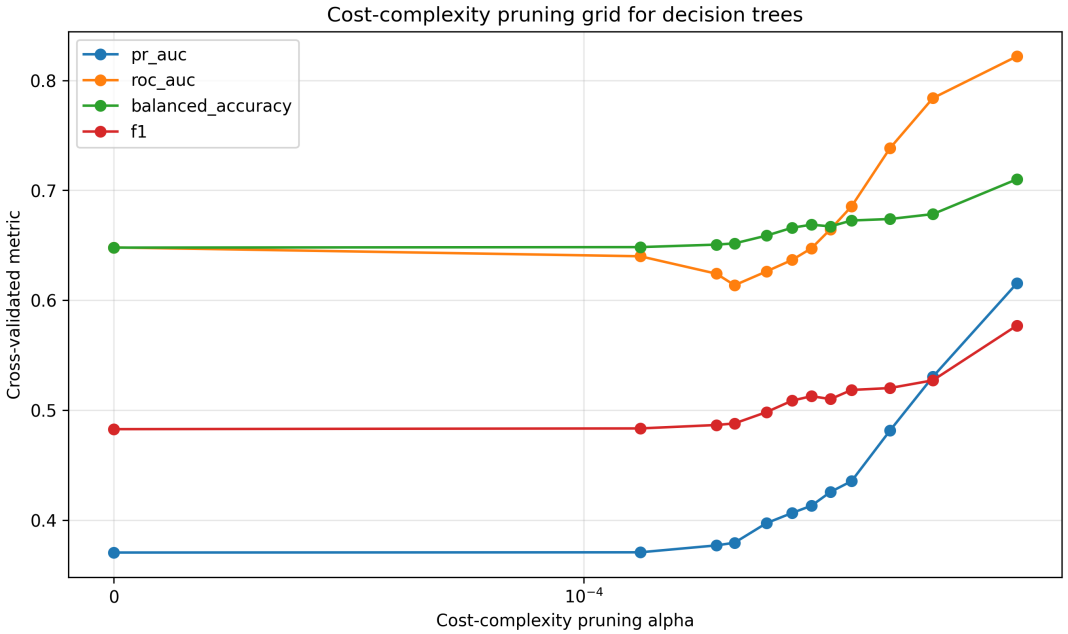

Figure 26: Cost-complexity pruning metrics for decision trees

The pruning result is useful because it demonstrates the central decision-tree bias-variance tradeoff. A larger tree can fit training irregularities, but a smaller pruned tree can generalize better. In this dataset, however, direct pre-pruning with a moderate maximum depth and minimum leaf size gives the strongest single-tree ranking result.

9.8 Model comparison

Table 37 compares the main decision-tree variants. The selected model is a Gini tree with `max_depth` = 6, `min_samples_split` = 25, `min_samples_leaf` = 10, and `ccp_alpha` = 0. It has PR-AUC approximately 0.628 and ROC-AUC approximately 0.824. It improves sharply over the default unrestricted tree, whose PR-AUC is only approximately 0.371.

Model	Accuracy	Bal. Acc.	Precision	Recall	Specificity	F_1	ROC-AUC	PR-AUC
Selected pre-pruned tree	0.789	0.701	0.624	0.514	0.888	0.564	0.824	0.628
Best cost-complexity-pruned tree	0.790	0.710	0.620	0.540	0.880	0.577	0.822	0.615
Decision stump	0.735	0.500	0.000	0.000	1.000	0.000	0.726	0.413
Default unrestricted tree	0.725	0.648	0.482	0.484	0.812	0.483	0.648	0.371

Table 37: Cross-validated decision-tree model comparison on the training set

The corresponding positive-first confusion-matrix counts are shown in Table 38. At the default threshold, the selected pre-pruned tree detects 769 of the 1495 churners and misses 726. It flags 463 non-churners as churn risks and correctly classifies 3676 non-churners.

Model	TP	FN	FP	TN
Selected pre-pruned tree	769	726	463	3676
Best cost-complexity-pruned tree	807	688	495	3644
Decision stump	0	1495	0	4139
Default unrestricted tree	723	772	777	3362

Table 38: Positive-first confusion-matrix counts for decision-tree models

The decision stump has ordinary accuracy equal to the no-churn majority baseline and predicts no churn at the default threshold. Nevertheless, its ROC-AUC is above 0.7, because the stump still assigns different leaf probabilities and therefore contains some ranking information. This illustrates why hard classification metrics and ranking metrics answer different questions.

Compared with previous model families, the selected tree is useful but not dominant. It is much better than the default unrestricted tree and stronger than Naive Bayes by PR-AUC, but it remains below logistic regression from Section 6. Its PR-AUC is close to selected k-nearest neighbours from Section 7. This supports continuing to tree ensembles later: a single tree captures meaningful nonlinear churn structure, but its instability and coarse leaf-based scoring limit its standalone performance.

9.9 Threshold behaviour

Figure 27 shows the threshold tradeoff for the selected tree. Lower thresholds increase recall but also create many more false positives. Higher thresholds improve precision and specificity, but recall falls.

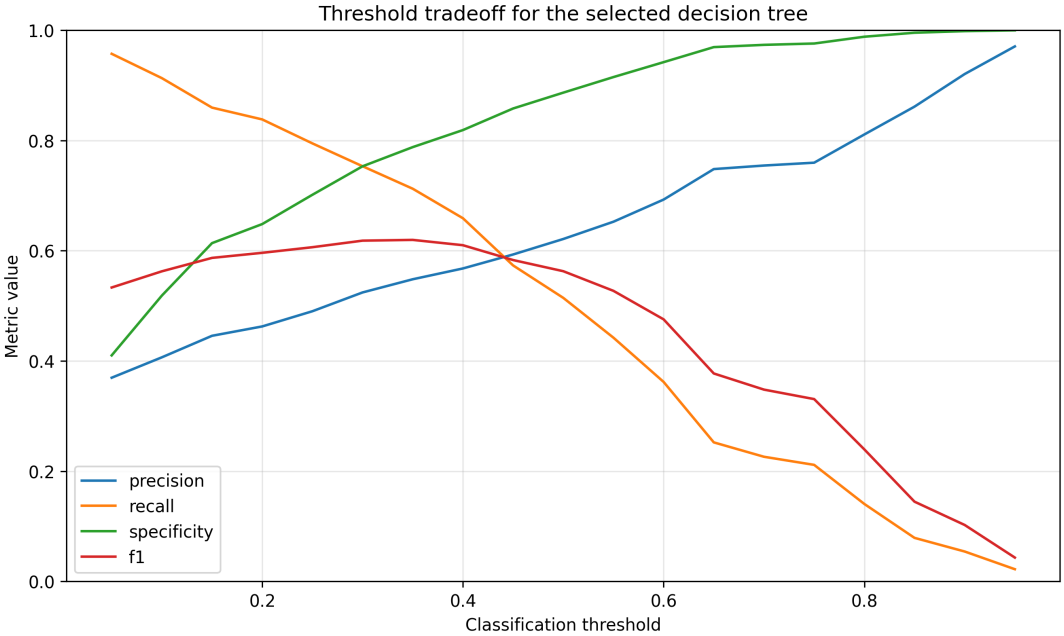

Figure 27: Threshold tradeoff for the selected decision tree

At threshold 0.30, the selected tree has precision approximately 0.524, recall approximately 0.753, specificity approximately 0.753, and F_1 approximately 0.618. At threshold 0.35, precision rises to approximately 0.548, recall is approximately 0.712, specificity is approximately 0.788, and F_1 is approximately 0.620. At the default threshold 0.50, precision is approximately 0.621, recall falls to approximately 0.514, specificity rises to approximately 0.887, and F_1 is approximately 0.563. This confirms again that the default threshold is not necessarily the best operating point for churn intervention.

9.10 ROC and precision-recall curves

Figure 28 shows the ROC curve for the selected tree. The ROC-AUC is approximately 0.824, indicating useful ranking ability.

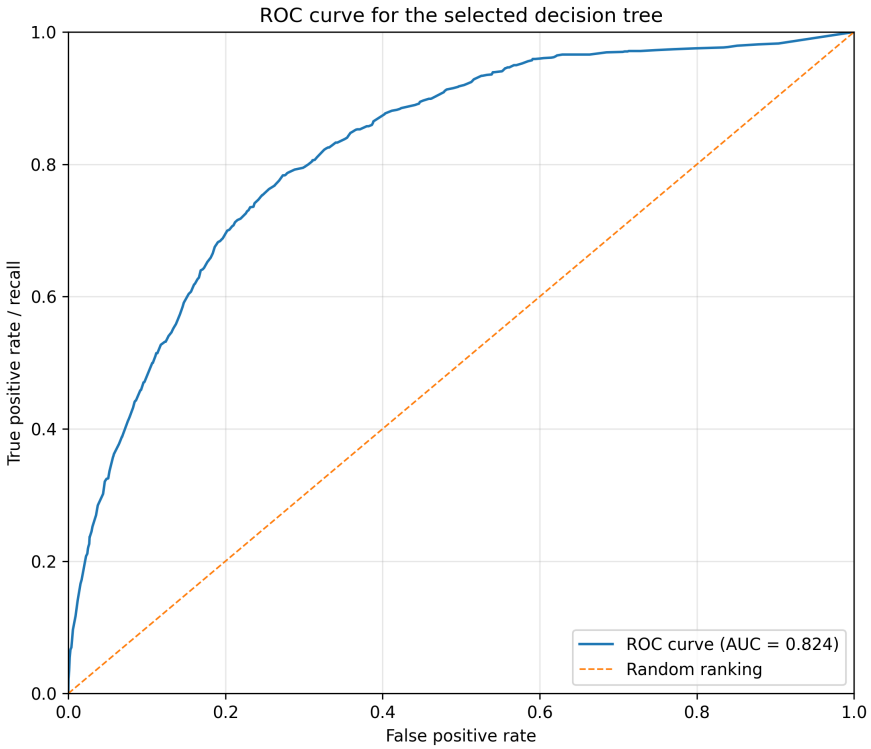

Figure 28: ROC curve for the selected decision tree

Figure 29 shows the precision-recall curve. The selected tree has PR-AUC around 0.63, well above the positive-rate baseline of approximately 0.265. This means that the tree ranks churners above non-churners much better than random ordering, even though its leaf-based probabilities produce a more stepwise ranking than smoother probabilistic models.

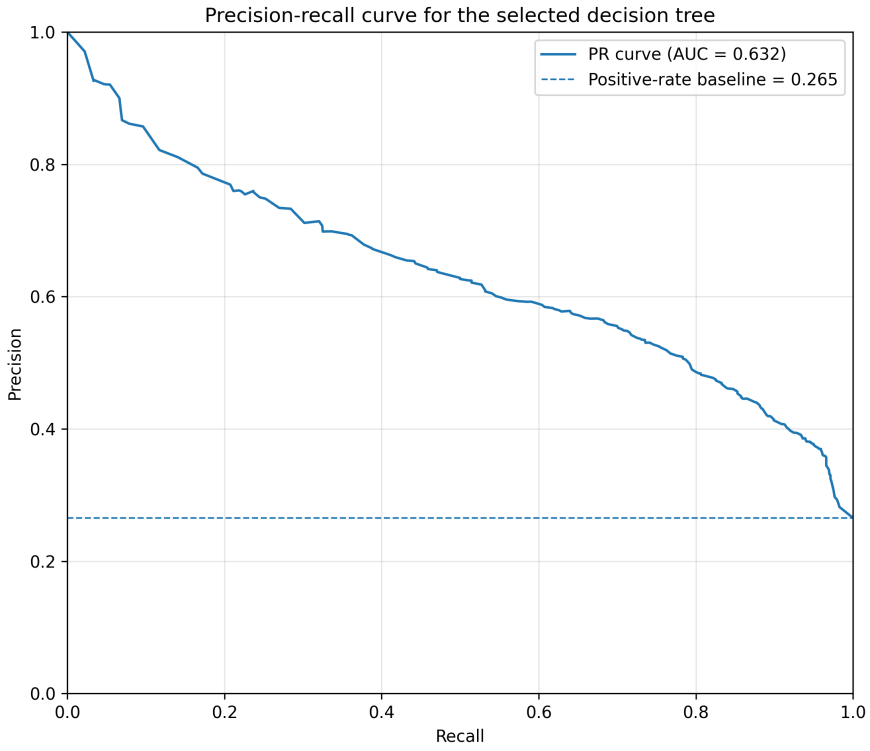

Figure 29: Precision-recall curve for the selected decision tree

9.11 Interpretation of the selected tree

The selected tree was refitted on the full training set only for interpretation. This refit is not used to estimate performance. Figure 30 shows the top levels of the fitted tree. The first split is on `Contract_Month-to-month`, which is consistent with earlier EDA and logistic-regression results. Month-to-month contracts are a central churn-risk indicator.

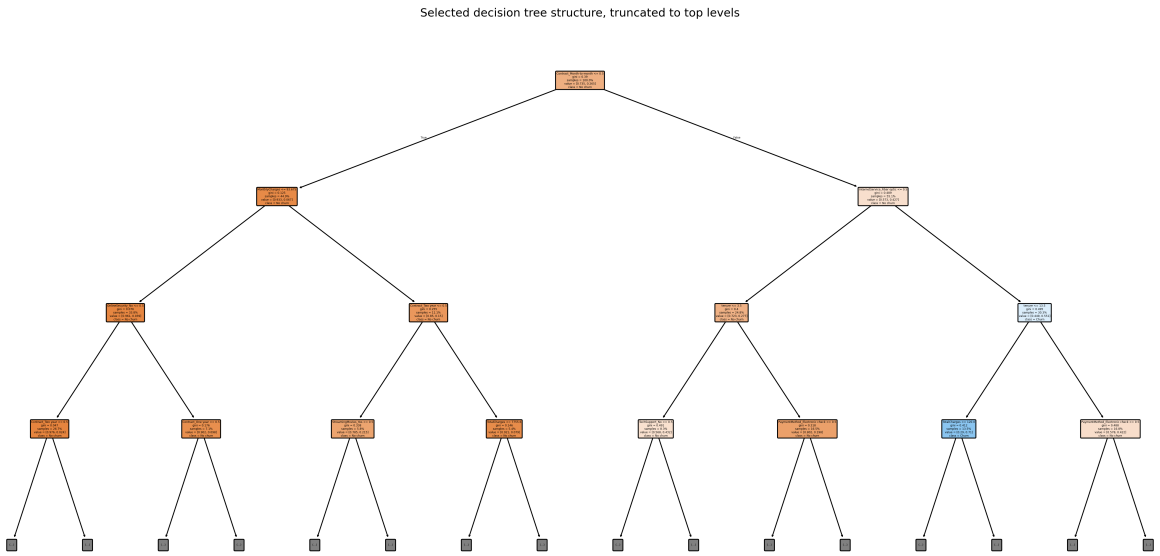

Figure 30: Selected decision-tree structure, truncated to the top levels

Figure 31 shows impurity-based feature importances. The dominant feature is `Contract_Month-to-month`, followed by `InternetService_Fiber optic`, `tenure`, `TotalCharges`, and `MonthlyCharges`. These variables are plausible churn drivers and align with previous sections.

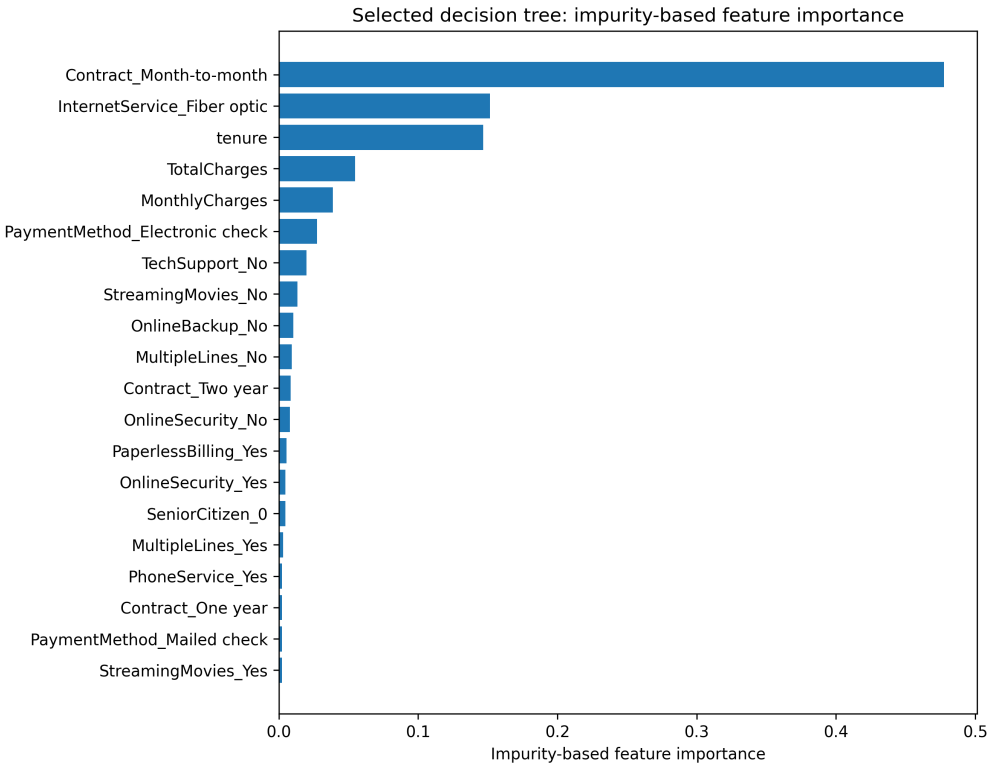

Figure 31: Impurity-based feature importance for the selected decision tree

The feature-importance plot should be interpreted cautiously. Impurity-based importances reflect how much splits reduce impurity inside the fitted tree. They are not causal effects, and they can be influenced by feature cardinality, correlated predictors, and the particular tree structure. Still, they are useful for understanding which variables the selected tree uses most heavily.

9.12 Summary

The decision-tree section demonstrates the strengths and weaknesses of single-tree classification. A decision tree provides interpretable rule-based nonlinear classification and naturally captures interactions between features. However, the default unrestricted tree overfits and performs poorly in ranking metrics. Regularization is therefore essential.

The selected single tree is a pre-pruned Gini tree with depth 6, minimum split size 25, and minimum leaf size 10. It reaches development-stage PR-AUC around 0.628 and ROC-AUC around 0.824. This is a substantial improvement over the default unrestricted tree and a meaningful result for a transparent nonlinear model, but it does not overtake logistic regression in the current development-stage comparison.

This outcome prepares the next modelling stages. Single trees are unstable, but they are the building blocks for bagging, random forests, and boosting. The next tree-based sections can therefore ask whether aggregating many trees improves the variance, ranking quality, and robustness limitations observed here.

10 Bagging and Random Forests

This section evaluates bagging-based tree ensembles for churn prediction. The previous section showed that a single regularized decision tree can capture useful nonlinear structure, but a single tree remains unstable because small data changes can alter the selected splits. Bagging and random forests address this weakness by fitting many trees and averaging their predictions.

The same development discipline is used as in the previous modelling sections. The held-out test set is not used. All reported results are based on stratified cross-validation inside the training set, using out-of-fold predictions where possible. The comparisons in this section are therefore development-stage comparisons of tree-ensemble variants. They identify representative ensemble candidates within the tried configurations, but they are not final test-set performance claims.

10.1 From one tree to an ensemble

Let a fitted decision tree produce a churn-risk score

$$\widehat{p}(x) = \widehat{P}(Y = 1 \mid X = x).$$

A single tree estimates this probability from the empirical churn rate in the terminal leaf reached by customer x. Because each split is chosen greedily and because terminal leaf proportions can be sensitive to the training sample, single trees often have high variance.

An ensemble reduces this instability by fitting many trees and averaging their predictions. If B trees are fitted and tree b produces score $\widehat{p}_b(x)$, the ensemble score is

$$\widehat{p}_{\text{ens}}(x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{p}_b(x).$$

The corresponding hard prediction at threshold t is

$$\widehat{y}(x; t) = \mathbb{1}\{\widehat{p}_{\text{ens}}(x) \geq t\}.$$

Thus, the ensemble remains a probability-scoring classifier. Ranking metrics such as ROC-AUC and PR-AUC can be computed from $\widehat{p}_{\text{ens}}(x)$, while hard classification metrics depend on the chosen threshold.

10.2 Bagging of decision trees

Bagging, short for bootstrap aggregation, fits each tree on a bootstrap-resampled version of the training data. For each ensemble member $b = 1, \ldots, B$, a bootstrap sample

$$\mathcal{D}^{*(b)} = \{(x_i^{*(b)}, y_i^{*(b)})\}_{i=1}^{n_b}$$

is drawn with replacement from the original training data. A decision tree h_b is fitted on this resampled dataset. Predictions are then averaged across all trees.

The main idea is variance reduction. Averaging many noisy predictors can be much more stable than using one noisy predictor. However, the reduction depends on how correlated the tree predictions are. If all trees make almost the same errors, averaging helps only a little. If the trees are diverse but still individually useful, averaging helps more.

In the bagged-tree variant used here, the base learner is a `DecisionTreeClassifier` inside scikit-learn's `BaggingClassifier`. The model uses bootstrap sampling across observations, while each tree can still consider all available features when choosing splits.

10.3 Random forests as a bagging-based extension

A random forest is a specialized bagging-based tree ensemble. Like plain bagging, it fits many trees on bootstrap-resampled training datasets. The additional random-forest idea is feature subsampling at each split. Instead of allowing every split to search over all p available features, each split considers only a random subset of features.

This extra randomness is designed to reduce correlation between trees. If one strong predictor dominates the split search, many bagged trees may look similar. Random feature subsampling forces different trees to explore different predictors and interactions. The ensemble average can then benefit more from diversification.

The distinction in this section is therefore not between unrelated model families. The comparison is between two tree-ensemble variants:

- **Plain bagged trees**: bootstrap-resampled decision trees where each split can consider all features.
- **Random forests**: bootstrap-resampled decision trees with additional random feature subsampling at each split.

10.4 Out-of-bag observations

Bootstrap sampling leaves out some observations from each tree's bootstrap sample. For a given tree b, observations not included in $\mathcal{D}^{*(b)}$ are called out-of-bag observations for that tree. These observations can be used to form out-of-bag predictions, because they were not used to train that particular tree.

Out-of-bag diagnostics are useful for tree ensembles because they provide an internal validation signal. In this project, however, out-of-bag scores are treated as supplementary diagnostics rather than as the main selection criterion. The main development comparisons still use the same stratified cross-validation framework as the earlier model sections, so that model-family results remain comparable.

10.5 Hyperparameter tuning strategy

Several valid strategies exist for hyperparameter tuning. Examples include fixed grid search, random search, Bayesian or Optuna-style optimization, successive halving, repeated cross-validation, and nested cross-validation. These strategies answer slightly different practical questions and have different computational costs.

This section uses transparent fixed grids. This choice is deliberate. The purpose here is model-family learning and controlled development-stage comparison, not final exhaustive optimization. A fixed grid makes it easy to see how the number of trees, bootstrap sample size, tree depth, leaf size, and feature subsampling affect performance. Later, after all model families have been studied, stronger training-only comparison procedures such as repeated cross-validation, paired comparisons, nested cross-validation, or more adaptive search can be used for serious final candidates.

The selected rows in this section should therefore be interpreted as the strongest configurations within the tried development grids, not as globally optimal hyperparameters and not as statistical proof of superiority.

10.6 Experimental design

The experiments compare four relevant models:

- the selected single decision tree from Section 9;
- a plain bagged-tree ensemble selected from a fixed development grid;
- a default random forest with 100 trees;
- a random forest selected from a fixed development grid.

PR-AUC remains the primary development ranking metric because churn is the minority class and positive-class retrieval is central to the project. Balanced accuracy and F_1 are retained as secondary threshold-dependent summaries. The final model selection stage will later revisit the strongest candidates across model families using a more careful training-only comparison.

10.7 Bagging grid results

Figure 32 shows the bagged-tree grid from the perspective of PR-AUC. The results are stable across the evaluated ensemble sizes. Increasing the number of trees from 50 to 100 and 200 produces only small changes, which indicates that the ensemble has already largely stabilized within this range.

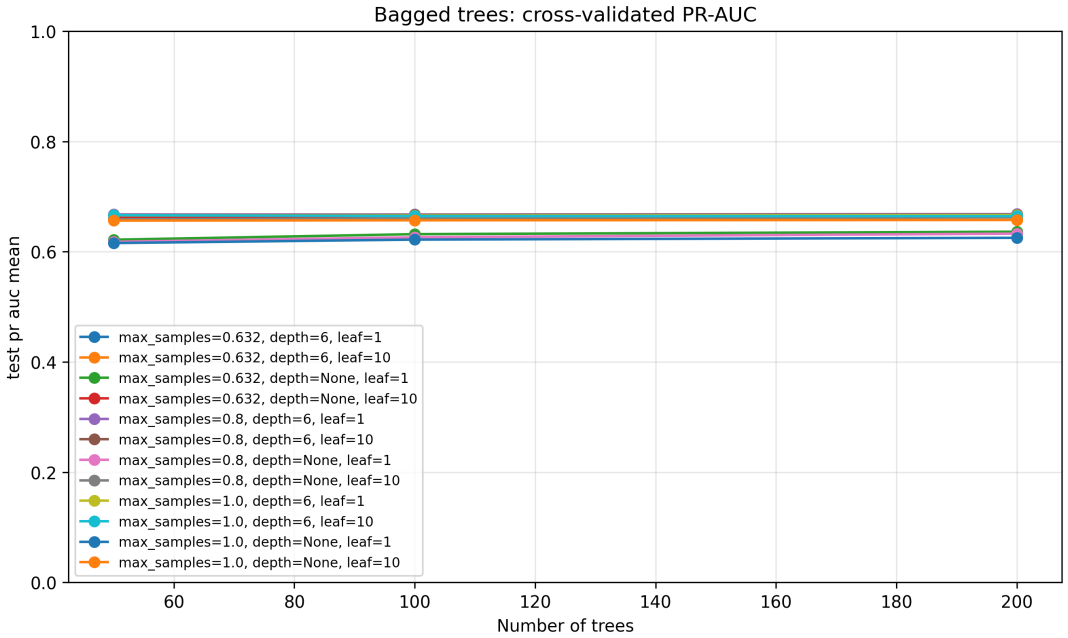

Figure 32: Bagged trees: cross-validated PR-AUC by number of trees

The best bagged-tree candidate in the fixed grid uses 200 trees, `max_samples` = 0.8, base-tree maximum depth 6, and base-tree `min_samples_leaf` = 1. Its grid-level mean PR-AUC is approximately 0.668. The small differences across neighbouring configurations should not be overinterpreted. The more important result is that bagging improves the single-tree ranking performance by averaging many bootstrapped trees.

Figure 33 shows the same bagging grid using balanced accuracy. The pattern is also flat across ensemble sizes, with values around 0.70. This supports the same interpretation: within the tried range, the exact number of trees matters less than the move from one tree to an averaged ensemble.

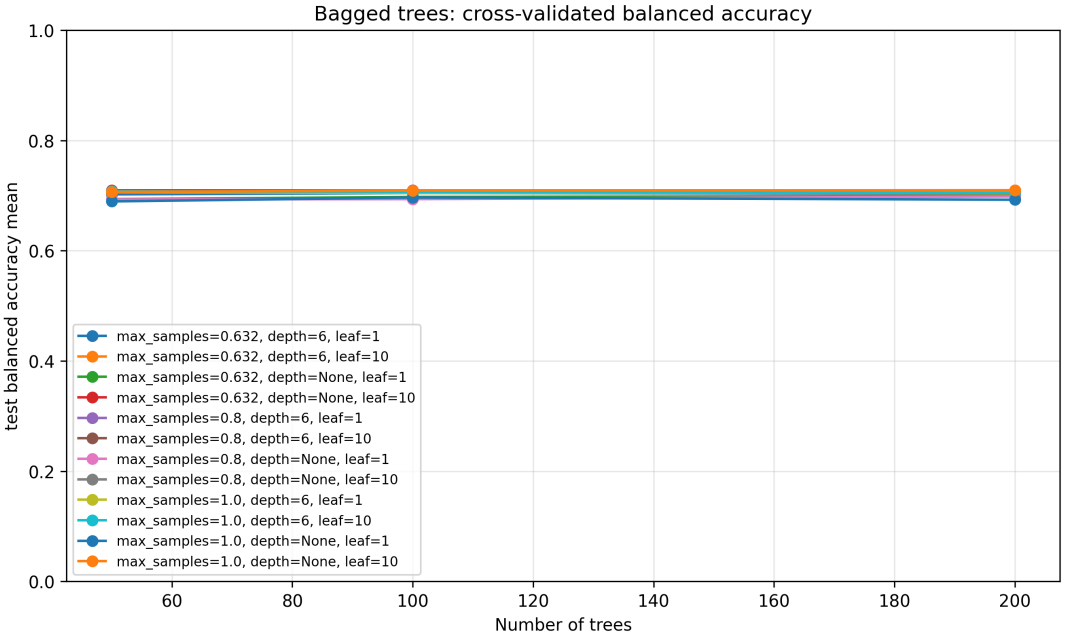

Figure 33: Bagged trees: cross-validated balanced accuracy by number of trees

10.8 Random-forest grid results

Figure 34 shows the random-forest grid from the perspective of PR-AUC. The strongest random-forest candidate uses 200 trees, maximum depth 10, `min_samples_leaf` = 10, and `max_features` = `sqrt`. Its grid-level mean PR-AUC is approximately 0.664.

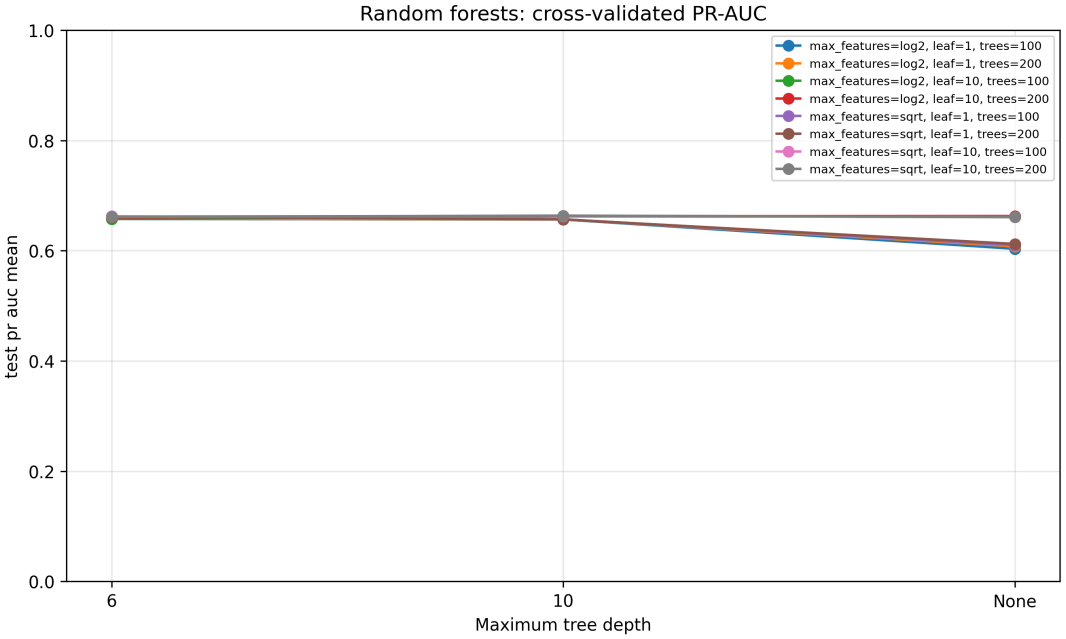

Figure 34: Random forests: cross-validated PR-AUC by maximum tree depth

The unrestricted-depth random-forest settings are weaker by PR-AUC than the depth-6 and depth-10 settings. This is consistent with the general tree-regularization pattern already seen in Section 9: even inside an ensemble, overly flexible trees can add noisy splits that do not improve validation ranking performance.

Figure 35 shows the random-forest grid using balanced accuracy. The metric is again very stable across many configurations. The depth-10 settings are slightly stronger than depth 6 and unrestricted depth, but the differences are small.

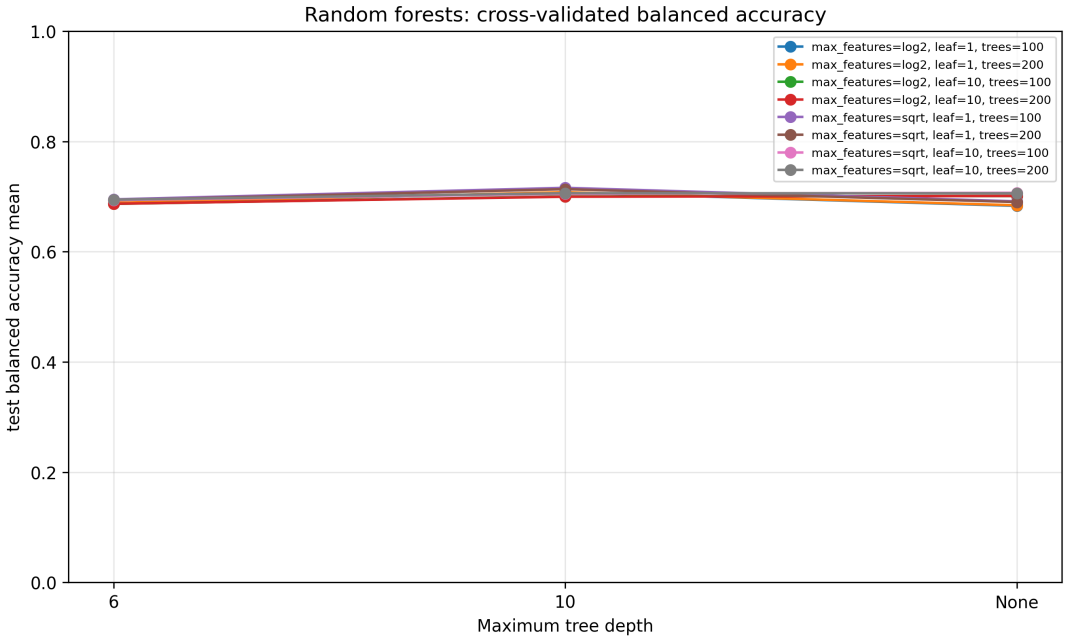

Figure 35: Random forests: cross-validated balanced accuracy by maximum tree depth

10.9 Model comparison

Table 39 compares the selected tree-ensemble candidates with the selected single decision tree and a default random forest. The selected bagged-tree ensemble improves PR-AUC from approximately 0.628 for the single tree to approximately 0.662. The selected random forest obtains PR-AUC approximately 0.660. Both ensembles also improve ROC-AUC from approximately 0.824 to approximately 0.846 or 0.847.

Model	Accuracy	Bal. Acc.	Precision	Recall	Specificity	F_1	ROC-AUC	PR-AUC
Best bagged trees	0.800	0.705	0.662	0.504	0.907	0.572	0.846	0.662
Best random forest	0.804	0.706	0.678	0.498	0.914	0.574	0.847	0.660
Selected single decision tree	0.789	0.701	0.624	0.514	0.888	0.564	0.824	0.628
Default random forest, 100 estimators	0.788	0.691	0.629	0.486	0.896	0.548	0.821	0.608

Table 39: Cross-validated bagging and random-forest model comparison on the training set

The main conclusion is not that plain bagging is statistically superior to random forests. The two selected ensemble variants are very close. Plain bagging has the slightly higher PR-AUC point estimate, while the random forest has slightly higher ROC-AUC, balanced accuracy, precision, specificity, and F_1. These differences are too small to interpret as statistical evidence that one tree-ensemble variant is truly better. The stronger conclusion is that bagging-based ensembles clearly improve over the single decision tree.

The corresponding positive-first confusion-matrix counts are shown in Table 40. At the default threshold, the selected bagged trees detect 753 of the 1495 churners and produce 385 false positives. The selected random forest detects 744 churners and produces 354 false positives. Thus, the selected random forest is slightly more conservative at the default threshold, with fewer positives flagged overall.

Model	TP	FN	FP	TN	Pred. positive rate
Best bagged trees	753	742	385	3754	0.202
Best random forest	744	751	354	3785	0.195
Selected single decision tree	769	726	463	3676	0.219
Default random forest, 100 estimators	727	768	429	3710	0.205

Table 40: Positive-first confusion-matrix counts for bagging and random forests

10.10 Out-of-bag diagnostics

Table 41 reports out-of-bag diagnostics for the selected ensemble candidates. The out-of-bag PR-AUC values are close to the cross-validated PR-AUC values, which is reassuring. The selected bagged-tree ensemble has out-of-bag PR-AUC approximately 0.660, while the selected random forest has out-of-bag PR-AUC approximately 0.663.

Model	OOB accuracy	OOB valid obs.	OOB ROC-AUC	OOB PR-AUC
Best bagged trees	0.801	5634	0.845	0.660
Best random forest	0.802	5634	0.847	0.663

Table 41: Out-of-bag diagnostics for selected bagging-based ensembles

These out-of-bag diagnostics support the earlier caution about the bagging versus random-forest comparison. The fixed-grid CV PR-AUC point estimate is slightly higher for plain bagging, while the out-of-bag PR-AUC is slightly higher for the random forest. This does not create a contradiction. It shows that the two ensemble variants are close enough that small changes in evaluation procedure can change their ordering.

10.11 Threshold behaviour

Figure 36 shows the threshold tradeoff for the selected PR-AUC representative, the bagged-tree ensemble. Lower thresholds increase recall but reduce precision and specificity. Higher thresholds reduce the number of flagged customers, raising precision and specificity but lowering recall.

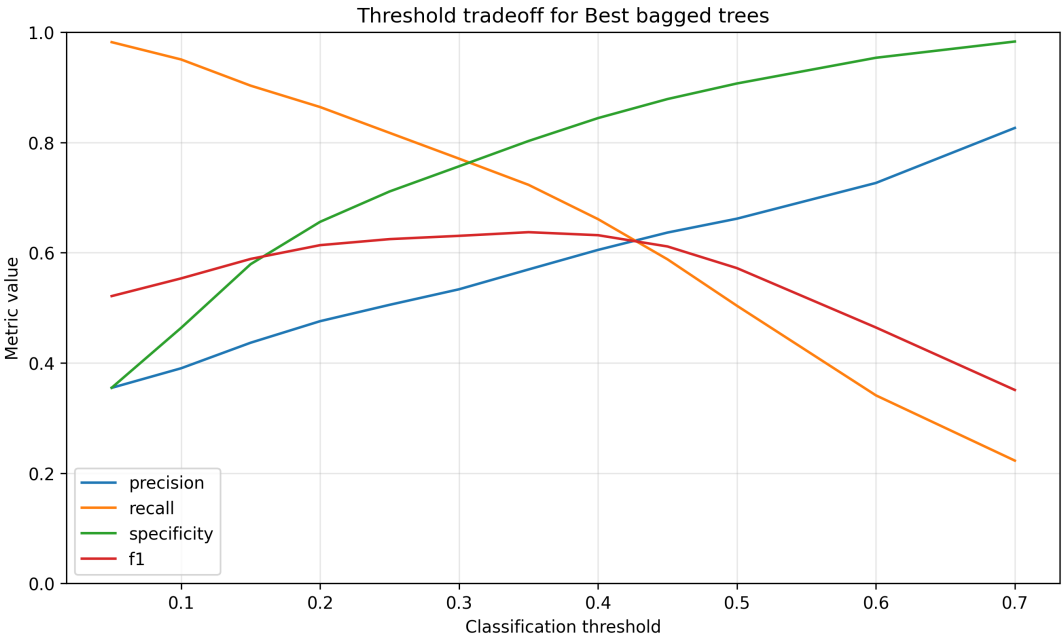

Figure 36: Threshold tradeoff for the selected bagged-tree ensemble

At threshold 0.30, the selected bagged trees have recall approximately 0.772, precision approximately 0.531, specificity approximately 0.762, and F_1 approximately 0.629. At threshold 0.40, recall is approximately 0.665, precision approximately 0.604, specificity approximately 0.843, and F_1 approximately 0.633. At the default threshold 0.50, recall falls to approximately 0.504, while precision rises to approximately 0.662. This again confirms that threshold choice is a separate operating decision and should not be confused with model ranking performance.

10.12 ROC and precision-recall curves

Figure 37 shows the ROC curve for the selected bagged-tree ensemble. The ROC-AUC is approximately 0.846, which improves over the selected single decision tree.

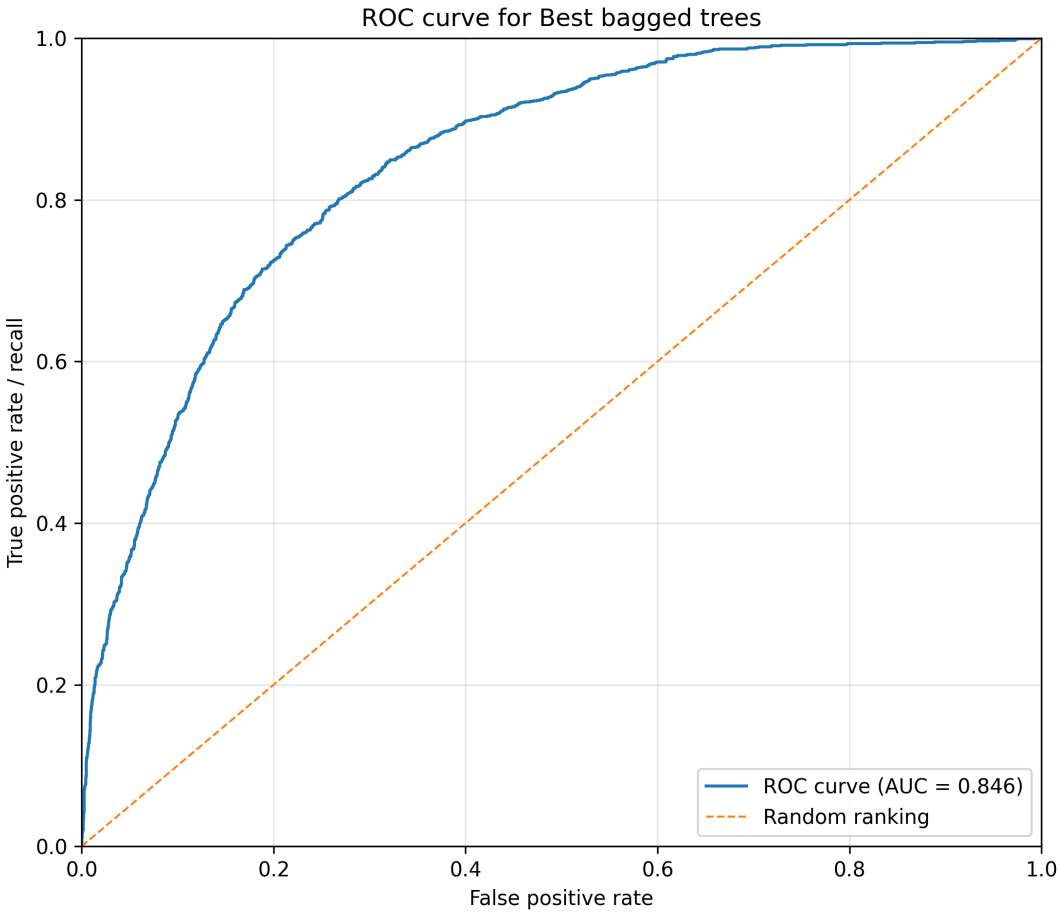

Figure 37: ROC curve for the selected bagged-tree ensemble

Figure 38 shows the corresponding precision-recall curve. The PR-AUC is approximately 0.661, well above the positive-rate baseline of approximately 0.265. The curve shows that high precision is possible only for the highest-risk customers, while precision gradually decreases as recall approaches one.

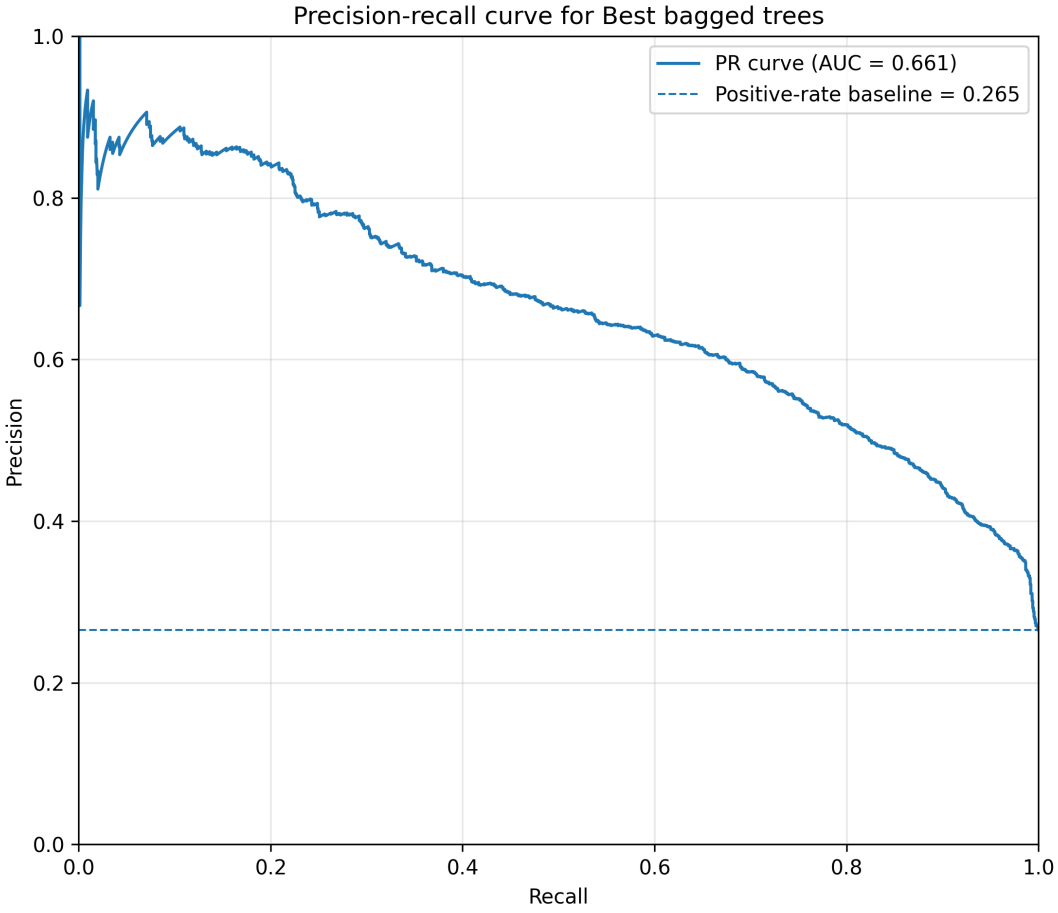

Figure 38: Precision-recall curve for the selected bagged-tree ensemble

10.13 Random-forest feature importance

Although the selected PR-AUC representative is the bagged-tree ensemble, the best random forest is retained as a close comparator and as a useful interpretation model. Figure 39 shows impurity-based feature importance for the best random forest.

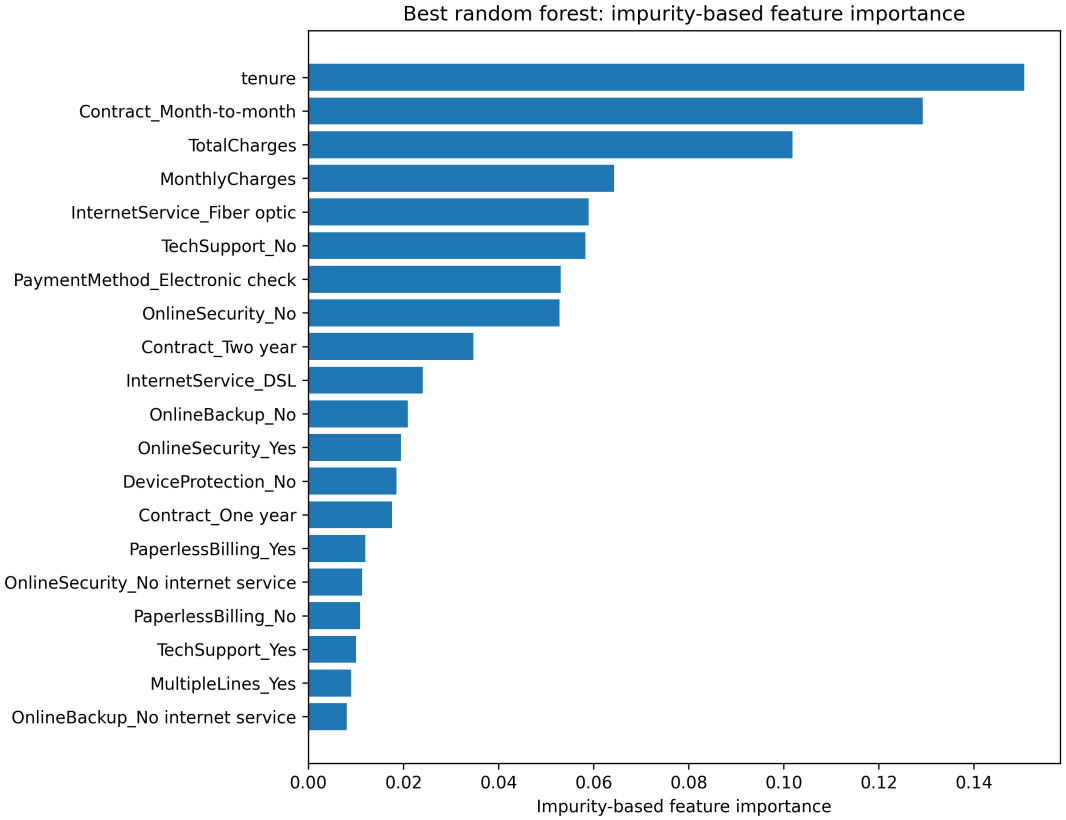

Figure 39: Best random forest: impurity-based feature importance

The most important variables are tenure, month-to-month contract status, total charges, monthly charges, fibre-optic internet service, lack of technical support, electronic-check payment, and lack of online security. This agrees with the main churn patterns found in earlier sections: contract structure, customer tenure, service type, and billing/payment behaviour are central to churn risk.

The feature-importance values should still be interpreted cautiously. Impurity-based importance measures how much splits on a feature reduce tree impurity across the forest. It is not a causal effect and it can be affected by feature correlation and the number of possible split points. In this project, feature importance is used as a descriptive diagnostic rather than as causal evidence.

10.14 Summary

Bagging-based tree ensembles improve meaningfully over the selected single decision tree. The selected bagged-tree ensemble raises PR-AUC from approximately 0.628 to approximately 0.662, while the selected random forest obtains PR-AUC approximately 0.660. Both ensembles also improve ROC-AUC to approximately 0.846 or 0.847.

The comparison between plain bagging and random forests is close. Plain bagging has the highest development-stage PR-AUC in the fixed grid, but the random forest is slightly stronger on several secondary metrics and on the out-of-bag PR-AUC diagnostic. These small differences should not be interpreted as statistical proof that one variant is superior. The appropriate conclusion is that both variants are promising tree-ensemble candidates.

This section therefore adds an important modelling step to the project. A single decision tree is interpretable but unstable. Bagging and random forests reduce that instability by averaging many trees. The next modelling section will move from averaging trees to boosting, where trees are

fitted sequentially so that later learners focus on mistakes made by earlier learners.

References

[1] Vrije Universiteit Amsterdam Machine Learning course notes. *Data pre-processing, Part 1: Missing values and outliers*. Used for the principles of not taking data at face value, distinguishing missing feature values from missing target labels, considering the production setting, and distinguishing corrupted values from natural extreme observations.

[2] Vrije Universiteit Amsterdam Machine Learning course notes. *Model Evaluation, Part 1: Experiments*. Used for train/validation/test discipline, cross-validation, the distinction between true performance and estimated performance, and the principle that held-out test data should not be reused during model development.

[3] Vrije Universiteit Amsterdam Deep Learning course notes. *Tools of the trade*. Used for the practical evaluation principles that test-set size controls metric uncertainty, validation data is used for development and hyperparameter tuning, test data is reserved for final reporting, leakage must be actively avoided, baselines should be used during development, hyperparameter tuning should balance performance and simplicity, and fair comparisons require comparable tuning effort.